Yuho: A Domain-Specific Language for Encoding the Singapore Penal Code as Executable Statute

Grammar, Tooling, and a 524-Section Proof of Concept

Gabriel Ong Zhe Mian
Independent
Singapore
gabrielzmong@gmail.com

Abstract

We present *Yuho*, a domain-specific language for encoding statutes as executable artefacts whose *encoding* is machine-checkable for structural and inter-section consistency (the verification stack does not adjudicate legal correctness; see §10), and a corresponding proof-of-concept encoding of the Singapore Penal Code 1871 (524 sections, 524 encoded, 524 passing parse-and-build, 524 carrying an author-administered fidelity stamp at the L3 tier — external-reviewer validation remains future work). The Yuho grammar models the structural primitives of criminal law: section headers and amendment lineage; elements partitioned into *actus reus*, *mens rea*, and circumstance with optional burden qualifiers; penalty combinators capturing cumulative, alternative, conditional, and nested punishment regimes; verbatim illustrations; Catala-style prioritised exceptions; and case-law anchors. A tree-sitter parser, a Python AST, eight transpilers (JSON, controlled English, LaTeX, Mermaid flowchart, Mermaid mindmap, Alloy, DOCX, Akoma Ntoso), a Language Server, a Model Context Protocol server, and a Z3/Alloy verification hookup turn an encoded statute into a queryable, diffable, formally-checkable object. We discuss fourteen grammar gaps surfaced by mass-encoding the Penal Code, the linting and fidelity-checking infrastructure built to keep encodings honest, and the engineering tradeoffs involved in modelling a single common-law jurisdiction in production-strength detail rather than a synthetic toy.

1 Introduction

Statutes are not free-form prose. A modern penal code is drafted to a tight internal grammar: numbered sections; enumerated elements; in-line illustration blocks; cross-cutting general exceptions; penalty clauses with named combinators. Lawyers learn to read this structure as if it were a typed program, and computer scientists have, since at least Sergot et al.'s 1986 encoding of the British Nationality Act [23], observed that the structure is amenable to direct symbolic representation. The persistent gap is not the recognition that statute looks like code — it is the practical absence of a representation expressive enough to encode an entire production statute without immediately falling back on free-text strings. §2 surveys the prior art across the logic-programming, markup, and modern-DSL traditions and explains why none of them target a common-law penal code at full coverage.

This paper investigates whether the persistent statute-encoding gap reduces, in practice, to a grammar gap, and presents *Yuho*, a domain-specific language designed by mass-encoding the entire Singapore Penal Code 1871. The headline number is engineering rather than methodological: 524 sections, 524 encoded, 524/524 passing parse-and-build (the L1+L2 tiers of our coverage harness), and 524 carrying an *author-administered* fidelity stamp at the L3 tier (§7.1; external-reviewer validation is future work, §10). Throughout, "machine-checkable" refers to the *encoding*: the verification stack checks structural and inter-section consistency of the encoded artefact, not the legal correctness of the statute itself. The interesting numbers sit underneath: every section that the grammar could not natively express is documented as a numbered *grammar gap* (G1–G14), and we report which were genuine grammar limitations, which were diagnostic issues misread as grammar issues, and which we resolved versus deferred.

Contributions. We make four contributions, the first of which is the methodological one we believe will travel beyond Yuho.

(1) **A gap-classification methodology.** A discipline for distinguishing genuine grammar limitations from fidelity issues masquerading as grammar issues, by classifying each encoding shortcoming according to the layer at which it is resolved (parser, linter, semantic resolver, or not-a-gap-on-re-test). We apply it to fourteen gaps surfaced during mass-encoding (G1–G14), preserving initial and final classifications honestly so readers can audit the three reclassifications (G2 and G7 reclassified to not-a-gap; G10 marked deferred because its remaining work is a semantic-resolver pass rather than a parser extension). The claim we advance is that conflating grammar gaps with linter gaps leads to grammar bloat with no corresponding gain in fidelity — and that the classification surfaces grammar-design feedback (e.g. a low-precision lint may indicate the grammar should disambiguate at the source) that a flat gap list does not.

(2) **The Yuho grammar.** A statute-shaped syntax in which the top-level construct is a `statute N { …}` block with first-class fields for definitions, elements, penalty, illustrations, exceptions, case law, parties, and amendment lineage. Elements carry a deontic type (`actus_reus` / `mens_rea` / `circumstance` / `obligation` / `prohibition` / `permission`), an optional burden qualifier (`burden prosecution` / `burden defence`) with a named proof standard, and optional causal (`causedBy`) and temporal (`precedes` / `during` / `after`) edges to other elements. Penalty clauses compose three top-level combinators (`cumulative` / `alternative` / `or_both`) plus a `when <ident>` conditional and a nested-combinator escape hatch. Exceptions form a Catala-style priority DAG with optional `defeats` edges to the elements they override.

(3) **A complete public encoding.** All 524 sections of the Singapore Penal Code 1871, version-controlled under `library/penal_code/`, with a per-section directory layout (`statute.yh`, `metadata.toml`, `test_statute.yh`). The encoding is canonical: the `.yh` file is the source of truth; all rendered formats are generated.
(4) **Tooling.** A tree-sitter parser, an immutable Python AST, eight transpilers (JSON, controlled English, LaTeX, Mermaid flowchart, Mermaid mindmap, Alloy, DOCX, Akoma Ntoso), a reference-graph resolver carrying Tarjan-SCC analysis plus the lam4-style `is_infringed` and Catala-style `apply_scope` cross-section predicates, a Z3-and-Alloy verification hookup, a fidelity-diagnostic pass that cross-checks encodings against the canonical scrape, a Language Server exposing hover / inlay-hint / completion / code-lens / diagnostics, a Model Context Protocol server for AI-driven encoding workflows, and a VS Code extension wrapping the Language Server.
(5) **Empirical findings on the 524-section sweep.** Per-gap incidence in the encoded corpus from `paper/methodology/gap_frequency.json` (G6/G8/G1 dominate, G13/G12 are rare); fidelity-diagnostic warning counts after Phase D (G4=98, fab-fine=48, fab-caning=2, G11=208); and a side-observation that the largest fidelity bucket (G11) is itself a low-precision heuristic whose 39.7% flag rate is a signal that a grammar-layer fix is the right long-term response. Diagnostic precision has not yet been hand-rated and we report this as a known gap (§7.4, §7.10).

Why criminal law and why Singapore? The Singapore Penal Code 1871 was selected for three pragmatic reasons. First, it is public: the canonical text is hosted on Singapore Statutes Online (SSO) [25] under permissive terms that allow programmatic ingestion. Second, it is modestly sized: 524 sections is small enough to mass-encode with manageable effort, and large enough to surface structural patterns that hand-picked toy examples would miss. Third, it inherits the Anglo-Indian drafting tradition shared with the Indian, Pakistani, Bruneian, and (with adjustments) Malaysian penal codes. Whether a grammar that fits the Singapore statute is portable across the wider Penal Code family is an empirical question we have not yet exhaustively tested; §9 reports the partial answer.

What this paper is not. It is not a claim that the encoded sections constitute legal advice, that the author-administered L3 fidelity stamp replaces a court's reading or a legal practitioner's judgement, or that Yuho is the right tool for every legal-encoding problem (regulatory corpora, contract drafting, treaty interpretation, and judgment summarisation each have grammars that Yuho does not natively model). It is a claim that the gap between a drafted penal section and an executable artefact is smaller than the existing literature suggests — and that an appropriately-shaped grammar dissolves a class of previously-reported encoding obstacles, leaving a residue that is tractable at full coverage for one statute family.

2 Background and Motivation

Statute encoding has a long history that, in the broadest stroke, decomposes into three traditions: the *logic-programming* tradition, the *markup-driven* tradition, and the recent *modern-DSL* tradition. Each makes different assumptions about what the artefact is for, who reads it, and what is preserved versus generated. Yuho draws from all three.

2.1 Logic-programming origins

The seminal work is Sergot et al.'s 1986 encoding of the British Nationality Act as a Prolog program [23]. The encoding showed that significant portions of nationality law could be evaluated as Horn clauses against case-style queries, and the paper articulated the now-standard caveats: open-textured terms must be left as free predicates; statutory defeats require careful negation-as-failure handling; and the gap between informal drafting and a Horn-clause semantics surfaces immediately at the boundary between rule and exception. Subsequent work by Sergot [22], Bench-Capon [3], and others extended the approach by adding argumentation frameworks and value-based reasoning, but did not change the fundamental shape: the statute is encoded as a flat rulebase, and structural primitives like illustrations or amendment lineage are either dropped or attached as comments.

The strength of this tradition is its executable semantics: a Prolog rulebase is queryable by construction. The weakness is its flatness. Penal code sections come with elements (which are typed, not just true/false), with illustrations (which are gold-standard test cases), with penalty clauses (which are not predicates but small algebras), and with general exceptions (which apply across multiple sections at once). A Horn-clause encoding can *simulate* all of these, but the simulation discards the structural shape of the statute, and a reader of the encoding cannot recover the statute that produced it.

2.2 Markup-driven codification

The second tradition, exemplified by Akoma Ntoso [17] and LegalRuleML [18], takes the orthogonal route: an XML schema designed for archival, exchange, and citation rather than execution. Akoma Ntoso, an OASIS standard, is now the lingua franca for intergovernment legislative exchange in much of Europe and Africa; LegalRuleML adds a layer for deontic norms (obligation, prohibition, permission) and isomorphic mappings to several rule languages.

These standards are excellent at what they do: they preserve every structural element of a drafted statute, including the parts most other representations discard (signatories, gazette references, amendment paths, multilingual variants, debate-history pointers). They are correspondingly verbose: a single section in Akoma Ntoso is several hundred lines of XML with extensive use of references and IDs, and the schema makes no commitment about what the elements *mean*. There is no notion that "actus reus" and "mens rea" are different kinds of element, that a penalty has internal algebra, or that exceptions have priority. The schema is a careful skeleton; the semantics is a separate problem.

For Yuho, this tradition is a constructive constraint rather than a target. Akoma Ntoso defines what every legislative document ought to preserve, and Yuho should be at least as faithful on the structural axis. Akoma Ntoso deliberately stops where semantics begins, and Yuho deliberately starts there. We expect Yuho-to-Akoma-Ntoso to be a future transpile target rather than a competing standard.

2.3 Isomorphism and the source-of-truth question

A recurring theoretical question across this body of work is whether the formal representation should mirror the textual structure of the source statute clause-for-clause (*isomorphism*, after Bench-Capon & Coenen [4]) or normalise it into whatever shape the formal logic prefers. Isomorphism privileges traceability: a statute clause and its encoding stand in one-to-one correspondence, so amendments propagate cleanly and auditors can read the encoding as the statute. The well-known cost is that some formal-logic-friendly normalisations (Skolemisation, Horn-clause flattening, scope-call wrapping) become non-options. Yuho sits firmly on the isomorphism side: a `statute N { …}` block is meant to be readable as the underlying section, and the penalty/exception/illustration primitives are deliberately statute-shaped rather than logic-shaped. The conclusion's "statute-as-source-of-truth" framing (§12) is this isomorphism position; the standard objections (open-textured terms cannot be mirrored without loss, normalisation tensions are real) apply, and we acknowledge them rather than try to dissolve them.

2.4 Deontic logic and burdens of proof

The deontic-logic tradition originates with von Wright [28] and is consolidated in the Handbook of Deontic Logic and Normative Systems [11]. Standard deontic logic (SDL) treats *obligation*, *prohibition*, and *permission* as modal operators with well-known interactions (prohibition(φ) $\equiv$ obligation($\neg\varphi$), contrary-to-duty paradoxes, etc.). LegalRuleML (above) carries these operators as first-class markup; defeasible-deontic-logic refinements (Governatori [12] and successors) handle permission/obligation interactions in machine-executable form. Yuho's `obligation` / `prohibition` / `permission` element kinds are syntactic positions that *name* the deontic role of an element; they do not carry the full SDL semantics, and we describe the gap explicitly in §3.4. The burden-of-proof qualifiers (`burden prosecution` / `burden defence` with named proof standards) are similarly labels rather than the procedural notions analysed in Prakken & Sartor [21]. Closing either gap is future work; the present paper claims only the structural surface, not the formal-deontic semantics.

2.5 Modern DSLs

The third tradition, dating roughly from the late 2010s, builds focused domain-specific languages for sub-areas of law where the underlying structure is regular enough to encode directly.

Catala. Catala [15], designed by Merigoux, Chataing, and Protzenko at INRIA for the French tax code, makes *prioritised default rules* the central construct. A Catala scope declares a base rule plus a prioritised list of exception rules; lower priority rules apply only when no higher-priority rule fires. This matches French civil-law drafting almost verbatim. Catala compiles to OCaml and Python and ships proofs of soundness for its rewriting system.

For Yuho, Catala's prioritised-default pattern is the right model for penal-code exceptions. Chapter IV of the Singapore Penal Code (sections 76 to 106) sets out general exceptions — mistake of fact, accident, self-defence, infancy, insanity — that defeat the elements of an otherwise-applicable offence. Each general exception comes with its own applicability conditions, and they have a partial priority order (self-defence, for example, has narrower scope than infancy). We adopt the priority semantics directly, in the form of a Catala-style DAG compiled into Z3 prioritised rewriting (§3.6). We disagree with Catala on top-level shape: Catala makes a *scope* (a named, callable function) the top-level construct. For penal code, the top-level construct is a *section*, which has fields (definitions, elements, illustrations, case law, parties) that a function-call shape does not naturally accommodate. Our position is that statute-shaped DSLs and scope-shaped DSLs solve different problems — and that scope composition (§8) belongs as an additional layer on top of statute blocks, not as a replacement.

lam4. lam4 [29] (from the SMU Centre for Computational Law) targets contracts and regulations with a deontic-logic core. Its distinctive primitive is the named-norm reference: a contract clause can predicate on whether another named norm has been infringed, via an `IS_INFRINGED` predicate. This composes cleanly with our planned G10 inter-section semantic hookup, which will let one Penal Code section predicate on whether another section's elements are satisfied (e.g. the abetment provisions in Chapter V refer to whichever offence is being abetted). lam4 targets contracts; Yuho targets penal code; the two grammars do not overlap, but the named-norm idea ports directly.

LexScript. LexScript [6] (MIT-licensed) is the closest tooling-shaped peer. It uses the `participle/v2` parser generator in Go (where Yuho uses tree-sitter) with a contract-shaped top level, compiles to multiple targets, and notably implements Tarjan strongly-connected-component analysis over the contract reference graph to detect cyclic cross-references and unreachable termination paths. We adopt the SCC analysis directly (planned, blocked on G10). LexScript's element model is thinner than ours: it has no burden qualifier, no causal-link edges between elements, and no Catala-style exception priority. The two systems also target different statute families: LexScript targets contracts, Yuho targets the penal code.

eFLINT.. eFLINT [26], developed at the University of Amsterdam, is a norm-based DSL with explicit acts, facts, and duties in a Hohfeldian frame. It is the closest peer to Yuho on the deontic-element-types axis: where Yuho carries `obligation` / `prohibition` / `permission` as element kinds, eFLINT carries duty / power / liability as norm shapes with executable semantics. The two systems target different domains (eFLINT: cross-sectoral norm specification; Yuho: penal code) but the overlap on deontic structure is direct, and a future Yuho-eFLINT mapping would be a natural way to inherit eFLINT's executable norm semantics for the deontic-element subsurface that Yuho currently treats syntactically.

DMN and Symboleo. The Decision Model and Notation [19] (OMG standard, v1.4) is the deployed benefits-and-eligibility decision DSL used by several legal-tech vendors; it provides FEEL expressions and a defined execution semantics for decision tables and decision-requirement diagrams. Symboleo [24] is a contract-specification language with formal semantics that sits between lam4 and Yuho on the contract-vs-statute axis. Neither targets penal-code structure, but both are part of the legal-DSL landscape against which Yuho should be positioned, and both are deployed beyond their authoring teams in a way Yuho is not.

LDOC.. LDOC [2] is a plain-text DSL for authoring professional documents that compiles to DOCX (TypeScript implementation, with an LSP server for editor integration). The source-of-truth posture matches Yuho's: a plain-text `.ldoc` file is canonical, and the rendered DOCX is a downstream artefact. The systems differ on target audience and structural commitment — LDOC is a general-purpose document-authoring DSL, where Yuho is statute-shaped — but the convergent decision to treat the plain-text source as canonical and DOCX as one of several rendered outputs validates Yuho's DOCX transpile target.

2.6 Verification infrastructure

Beyond DSLs, several lines of work import formal-methods machinery into legal reasoning. SMT solvers (notably Z3 [7]) check consistency of legal rule sets and decide queries about statutory quantities. Alloy [14] provides bounded model-checking over relational specifications, useful for enumerating element-combination counterexamples. Yuho's verification hookup uses both: Z3 for exception- priority validation and inter-section conflict checks, Alloy for bounded enumeration of element configurations. We discuss what these solvers can and cannot do for statute encoding in §10.

2.7 Tooling for legal-DSL authors

The Language Server Protocol has matured into the de-facto integration point between editors and language tooling, and is used by every modern DSL listed above. The Model Context Protocol [1] is newer (2024) and provides a parallel surface for AI clients: tools the model can call, resources the model can read, and prompts the host can pre-load. We use both: the Language Server for human authors, the MCP server for AI-driven encoding workflows. Both surfaces wrap the same Python AST and analyser stack, so a diagnostic added to the linter is immediately visible to both audiences.

2.8 Why a new DSL?

The case for Yuho is not that any existing DSL is wrong — each is well-fit to its declared domain — but that no existing DSL targets a common-law penal code at full coverage. Catala has the right defaults but no penalty algebra and no illustrations. lam4 has the right deontic core but expects contracts, not penal sections. LexScript has the right parser-generator-driven shape but a thinner grammar and contracts as its target. Akoma Ntoso has structure without semantics. LDOC is plain-text-source-canonical like Yuho but is a general-purpose document-authoring DSL, not statute-shaped.

The Singapore Penal Code 1871 was selected as the testbed for three reasons elaborated in §1: it is public, modestly sized, and inherits the Anglo-Indian drafting tradition shared with several other jurisdictions. The remainder of the paper presents the grammar that mass-encoding it produced, the tooling stack we built around the grammar, and the empirical findings from running both at full coverage.

3 Design

This section presents the Yuho grammar through a worked example, then unpacks each structural primitive in turn. We close with a catalogue of the fourteen grammar gaps that mass-encoding the Singapore Penal Code surfaced, classified into ten genuine grammar limitations (all resolved by extending the parser) and four diagnostic-layer issues (resolved by adding fidelity linters; see §5.6).

3.1 A worked example: section 415

Section 415 of the Penal Code defines the offence of cheating. The canonical drafting establishes deception as the foundational act, requires either fraudulent or dishonest mens rea, requires that the deception induces one of three specific consequences (delivery of property, consent to retain property, or doing/omitting an act the victim would not otherwise have done), and requires that the inducement cause or be likely to cause damage to body, mind, reputation, or property. Three illustrations follow. Figure 1 shows the full Yuho encoding of the statute block (the file under `library/penal_code/s415_cheating/` also contains a helper `struct` and `match` expression for client-side reasoning, omitted here for brevity).

The encoding tracks the drafted statute one-for-one. Definitions reproduce the in-statute glossary. The element block is a typed conjunction with one disjunctive sub-group for the alternative mens rea. The penalty is a flat range over imprisonment and fine. Each illustration is a verbatim worked example with a stable label.

3.2 A harder worked example: section 302 (murder)

Section 415 is a deliberately simple introductory example: it has typed elements, a flat penalty, three illustrations, and no encoded exceptions. Several of the primitives we claim as distinctive — nested penalty combinators (G12), the `when` conditional (G9), the `caning := unspecified` sentinel (G14), and exception priority (G13) — do not fire on s415. Section 302 (murder) exercises three of them at once: its post-2012 penalty has the form "death *or* imprisonment for life, *and* shall also be liable to fine" — a nested combinator (`cumulative` wrapping an `alternative` branch) with a named `when` guard distinguishing the residual category in which the death penalty is available from the default life-imprisonment case.

The combinator tree (already in Figure 3) is what G12 purchased: before G12 landed, the same content required two sibling `penalty` blocks plus a free-text comment disambiguating the relationship between them. The `when residual_category` clause (G9) names the post-2012 condition under which death is reachable, separating it from the default life-imprisonment penalty without requiring conditional logic in the encoder's prose. The `fine := unlimited` sentinel (G8) encodes "and shall also be liable to fine" without inviting the encoder to fabricate a numeric cap (§5.6). Finally, the `exception` block with `defeats killing` ties a Chapter IV general exception to the specific element it overrides — the priority field places it in the DAG (G13) so that the compilation to Z3 prioritised rewriting (§3.6) fires this defeater before lower-priority exceptions. Cross-section references (e.g. `s300_limb` pointing into the s300 element graph) remain a free-text circumstance label in this example because the cross-section semantic resolver (G10) is fully landed: the resolver builds an adjacency-list reference graph over the encoded library and supports BFS reachability plus a Tarjan strongly-connected-components pass that surfaces structural cycles in the cross-reference graph (§7.3).

```
statute 415 "Cheating" effective 1872-01-01 {
  definitions {
    deceive      := "To cause a person to believe ...";
    fraudulently := "With intent to defraud another person";
    dishonestly  := "With intention of causing wrongful gain/loss";
    property     := "Any movable or immovable property";
    inducement   := "Causing a person to act based on deception";
  }

  elements {
    all_of {
      /// deception is the foundational act
      actus_reus  deception  := "Deceiving any person";
      any_of {
        /// per s25: intent to defraud
        mens_rea  fraudulent := "Fraudulently inducing the person";
        /// per s24: intent to cause wrongful gain/loss
        mens_rea  dishonest  := "Dishonestly inducing the person";
      }
      /// deliver property / consent to retain / do/omit an act
      actus_reus  inducement := "Inducing delivery of property, ...";
      circumstance harm      := "Causing or likely to cause damage ...";
    }
  }

  penalty {
    imprisonment := 1 year .. 7 years;
    fine         := $0.00 .. $50,000.00;
  }

  illustration example1 {
    "A intentionally deceives B into believing that
     a worthless article is valuable, and thus induces
     B to buy it. A cheats."
  }
  /* example2, example3 omitted for space */
}
```

Figure 1: Yuho encoding of *Penal Code s. 415 (Cheating)*. The `statute` block is the top-level construct; it has named field positions for definitions, elements, penalty, and illustrations, plus optional positions (not shown here) for exceptions, case law, parties, and subsections. The element block uses an `all_of` combinator with a nested `any_of` for the disjunctive mens-rea requirement. Doc comments (`///`) attach as `doc_comment` fields on the AST node they precede.

```
statute 302 "Murder" effective 1872-01-01
                     effective 2012-12-31 {
  elements { all_of {
    actus_reus  killing  := "Causing the death of a person";
    mens_rea    intent   := "Intent to kill or knowledge of likelihood";
    /// per s300 limbs
    circumstance s300_limb := "Falls within s300 (a)-(d)";
  } }

  penalty {
    cumulative {
      alternative when residual_category {
        death;
        imprisonment := life;
      }
      fine := unlimited;
    }
  }

  exception "private_defence" priority 1
    guard "private defence under sections 96-106"
    defeats killing;
}
```

Figure 2: Yuho encoding of s302 (Punishment for murder), illustrating the nested penalty combinator (G12), the `when` conditional (G9), the `fine := unlimited` sentinel (G8), and a Chapter IV general exception with a `defeats` edge.

The lam4-style `is_infringed(s299)` predicate that lets s107 abetment say "if s299 `is_infringed`, then ...", and the Catala-style `apply_scope(s299, facts)` composition that returns the inner scope's `overall_satisfied`, both ship at all three layers: tree-walking interpreter, library-level lint, and Z3 backend (where each call site translates to the target's `<sX>_conviction` Bool, declared lazily so cross-section references resolve to the same atom regardless of statute iteration order).

3.3 The statute block

The top-level construct of every Yuho file is one or more statute blocks. Each block carries a section number (with optional alphabetic suffix — e.g. s376AA — supported after gap G3), an optional title string, and one or more effective <date> clauses (G6). The body is a sequence of named-field positions: definitions, elements, penalty, illustration (repeatable), exception (repeatable), caselaw (repeatable), parties, and subsection (N) (repeatable, with nesting). All field positions are optional except that a section without any elements or subsections carries no semantic content and is rejected.

The choice to make statute the top-level construct, rather than a more abstract notion like Catala's *scope*, is deliberate. Penal sections come bundled with fields that a function-call shape does not naturally accommodate: illustrations are not function arguments, and case-law anchors are not return values. Treating the section as a struct with named fields matches the way the statute is drafted and the way a lawyer reads it.

3.4 Element decomposition

Every offence is encoded as a typed conjunction (or, for genuinely disjunctive offences, a typed disjunction) of *elements*. The element type system is the kind taxonomy:

- actus_reus — the prohibited act.
- mens_rea — the required mental state.
- circumstance — contextual conditions that must hold (time, place, status of victim, etc.).
- obligation / prohibition / permission — deontic operators for sections drafted as duties or prohibitions rather than offences proper.

Elements are grouped by combinator. The all_of combinator denotes conjunction (every member must hold); the any_of combinator denotes disjunction (at least one member must hold). Groups nest: the s415 example uses all_of at the top with a nested any_of for the alternative mens rea. Beyond the combinator, each element carries an optional *burden qualifier* — burden prosecution or burden defence, optionally with a named *proof standard* (beyond_reasonable_doubt, balance_of_probabilities, prima_facie) — and optional causal (causedBy <other_element>) and temporal (<element> precedes <element>, during, after) edges to other elements in the same statute. The causal edges proved necessary while encoding sections like s299 (culpable homicide), where the act and the death are distinct elements connected by a causal requirement that is itself a frequent litigation point.

3.5 Penalty combinators

Penal sections specify punishment as a small algebra over four primitives (imprisonment, fine, caning, death) plus three top-level combinators (cumulative, alternative, or_both), one conditional combinator (when <ident>), and a nested-combinator escape hatch.

Combinators. The default combinator is cumulative: all listed punishments apply. The alternative combinator selects one branch at sentencing. The or_both combinator, which the Penal Code uses pervasively ("shall be punished with imprisonment of either description for a term which may extend to N years, or with fine, or with both"), is a binary case where the sentencer may impose either or both. We treat or_both as a first-class primitive rather than a sugar over alternative, because the canonical drafting names it explicitly and the verification semantics differ.

Sentinels. Two sentinels avoid the temptation to fabricate numeric ranges where the statute is silent. The fine := unlimited sentinel (G8) encodes "with fine" clauses that name no upper bound; the caning := unspecified sentinel (G14) encodes "shall be liable to caning" clauses that name no stroke count. Without these sentinels, encoders are tempted to write fictitious caps ("$100,000" or "24 strokes") that the statute does not authorise — which a fidelity linter then catches as fabrication (§5.6).

Conditional branches. The penalty when <ident> construct (G9) names a conditional penalty branch. Sections where the punishment depends on a named factual condition — for example, s302 where the death penalty applies only when the offence falls within the limited residual category preserved by the 2012 amendments — become readable as a small case analysis rather than as embedded prose.

Nested combinators. A penalty of the form "A and (B or C)" — "death or imprisonment for life, and shall also be liable to fine" in s302, for example — requires composing combinators rather than flattening them. Gap G12 added a nested field on PenaltyNode that carries a sub-penalty with its own combinator, which lets a single penalty block express the full algebra. Before G12 this was a frequent encoder workaround: nineteen sections in the Penal Code used a two-sibling-block hack that we collapsed into nested form via a one-shot migration script. Figure 3 shows the resulting combinator tree for s302.

3.6 Exceptions as Catala-style defaults

Chapter IV of the Penal Code (sections 76 through 106) sets out the *General Exceptions* — mistake of fact, accident, judicial act, infancy, insanity, intoxication, consent, communication made in good faith, private defence, and others. These exceptions are cross-cutting: each one applies, in principle, to every offence in the Code, and they have a partial priority order (intoxication is narrower than insanity; private defence is narrower than mistake; and so on).

We encode each exception as a Yuho exception block with three fields: a condition string (the natural-language scope), an optional effect string (what changes when the exception applies), and a priority N integer (the rank in the DAG; lower numbers fire first). An optional defeats <element_name> edge ties the exception to the specific element it overrides.

The compilation target is Z3 prioritised rewriting in the style of Catala [15]: the base rule fires; lower-priority exceptions fire only when no higher-priority exception matches. This semantics is the right shape for general exceptions, which behave precisely as prioritised defaults rather than as flat negations. Figure 5 shows the priority DAG over a typical Chapter IV exception cluster.

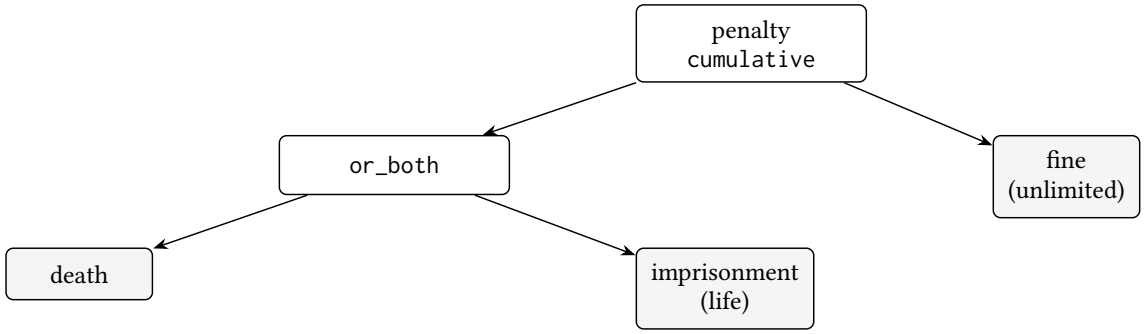

Figure 3: Nested penalty combinator for s302: a top-level `cumulative` block wrapping an `or_both` branch (death or imprisonment-for-life) plus an unconditional `fine := unlimited`.

```
exception "consent" priority 1
  guard "victim consents and exception 87 applies";
exception "self_defence" priority 2
  guard "private defence under sections 96-106"
  defeats inducement;
```

Figure 4: Two Chapter IV exception bindings on a host offence, encoded as priority-DAG nodes with a `defeats` edge.

3.7 Subsections and amendment lineage

A statute may carry nested `subsection (N) { …}` blocks (gap G5). Each subsection has its own definitions, elements, penalty, illustrations, and exceptions, and may itself contain further nested subsections. This matches the drafting reality of sections like s377BO (seven subsections) and s511 (three): the subsections are semantically distinct and frequently carry their own penalty regimes.

Amendment lineage is encoded by repeated `effective <date>` clauses on the `statute` header (gap G6). The Singapore Penal Code has been amended substantively at least four times since 1872 (notably 1985, 2007, 2012, and 2019); for sections introduced or materially rewritten by an amendment, the encoding carries both the original `effective 1872-01-01` clause and the amendment `effective <date>` clauses. The optional `repealed <date>` clause closes a section. Two further metadata clauses — `subsumes` and `amends` — record cross-section relationships at the header level, surfacing for reference-graph analysis (§8).

3.8 Annotations and metadata

The grammar reserves three lightweight annotation positions: `///` doc comments on most node positions (lifted into the AST as `doc_comment` fields, gap G1 fixed), file-level `/// @meta key=value` pairs (used for the canonical SSO source URL, the controlling Act number, and the amendment number), and a `/// @amendment [N/YYYY]` marker that ties an encoding to a specific gazette amendment. These annotations are recoverable from the AST without polluting the core grammar; they are read by the LSP hover handler, the MCP server, and the documentation transpilers.

3.9 The fourteen grammar gaps

Mass-encoding the Penal Code surfaced fourteen distinct grammar shortcomings, summarised in Table 1. We classify each as a real grammar limitation (ten items, all resolved by extending the parser) or a not-a-gap that the diagnostic layer catches (four items, resolved by adding fidelity linters). Two gaps initially classified as grammar limitations turned out, on closer inspection, not to be gaps at all (G2, G7); one (G10) is a not-a-gap on the grammar side but does require a separate semantic-resolver pass that remains future work.

The empirical takeaway from the gap catalogue is that the gap classification itself matters. A gap that surfaces as "the parser rejects this encoding" might be a parser bug (G1, G3, G5), a missing combinator (G8, G12), a missing sentinel (G14), or a fidelity issue masquerading as a grammar issue (G4, G11). Distinguishing these matters because the resolution cost differs by an order of magnitude — a new parser rule is a few lines in `grammar.js`, while a fidelity linter requires walking the AST against the canonical scrape — and conflating them leads to grammar bloat with no corresponding fidelity gain.

4 Formal semantics

§3 presents Yuho informally; this section pins the operational semantics for the fragment of the language the empirical claims rely on. We give small-step rules for element evaluation, prioritised-exception precedence in the Catala [15] default-logic style, the cross-section composition predicates `apply_scope` and `is_infringed`, and the penalty algebra. The body of this section is the two-page summary reviewers can follow without flipping to the appendix; the complete inference rules (including the elided cases for "mens rea" typed-intent variants, temporal-ordering constraints, and the burden-of-proof qualifier) appear in §10. We deliberately omit any mechanisation here — the soundness theorem of §6 rests on the pen-and-paper rules below; a Coq mechanisation is left as future work and noted in §10.

Table 1: The fourteen grammar gaps. "Initial class." is what the gap was called when first surfaced during mass-encoding; "Final class." is the resolution after closer inspection. Three gaps shifted class post-hoc: G2 and G7 were reclassified as not-a-gap once the underlying issues were re-tested or subsumed by G5; G10 stays a grammar concern in name but its remaining work is a semantic-resolver pass and we mark it deferred rather than fixed. "Freq." is the post-fix incidence in the encoded corpus from `paper/methodology/gap_frequency.json`; entries marked "—" have no post-fix incidence (G2 and G7 because they were not gaps; G10 because the semantic resolver is unimplemented). G8 is reported as two related counts: `fine := unlimited` sentinel uses (249) and `or_both` combinator uses (190). Discovery-order narrative is in `docs/researcher/phase-c-gaps.md`.

Gap	Initial class.	Final class.	Freq.	Artefact
G1	grammar	fixed	280	doc comments before `element_group`
G2	grammar	not-a-gap	—	colons in `///` parse fine after re-test
G3	grammar	fixed	23	multi-letter section suffix (s376AA, s377BO)
G4	grammar	lint	98	illustration-count vs canonical
G5	grammar	fixed	123	`subsection (N) { …}` nesting
G6	grammar	fixed	243	multiple `effective <date>` clauses
G7	grammar	not-a-gap	—	flat `definitions` suffices; G5 covers nesting
G8	grammar	fixed	249/190	`or_both` / `alternative` / `cumulative`; `fine := unlimited`
G9	grammar	fixed	49	`penalty when <ident>` conditional branches
G10	grammar	deferred	—	cross-section semantic resolver — grammar already parses `referencing` / `subsumes` / `amends`
G11	grammar	lint	208	`all_of` vs "or" connective check
G12	grammar	fixed	11	nested penalty combinators
G13	grammar	fixed	1	exception-priority DAG with Z3 prioritised rewriting
G14	grammar	fixed	31	`caning := unspecified` sentinel

4.1 Preliminaries

Syntactic domains. A Yuho *module* M is a multiset of *statutes*; a statute $S = \langle n, D, Es, P, Xs, Subs \rangle$ where n is its section number (a string like `300` or `376AA`), D is a set of named definitions, Es is a tree of element nodes and groups, P is an optional penalty algebra term, Xs is a list of exception nodes carrying priority and `defeats` edges, and $Subs$ is a sequence of subsection sub-statutes.

An *element* e is a triple $\langle \kappa, x, \rho \rangle$ where $\kappa \in \{\text{actus_reus}, \text{mens_rea}, \text{circumstance}\}$ tags the doctrinal kind, x is the element's identifier, and ρ is its free-text description (semantically opaque to the rules below). An *element group* $g = \langle c, \vec{m} \rangle$ has a combinator $c \in \{\text{all_of}, \text{any_of}\}$ and an ordered sequence of member nodes (each a leaf e or a nested group). *Exception* $\xi = \langle \ell, \gamma, \delta, \pi \rangle$ has a label ℓ, a guard expression γ (a boolean predicate over facts), an optional set δ of sibling-exception labels it defeats, and an optional priority $\pi \in \mathbb{N}$ (lower is higher-priority).

Semantic domains. A *fact pattern* F is a finite map from element identifiers to truth values, augmented with the typed constants the encoded library uses (Intent enums, money quantities, durations). The *statute environment* Σ is a finite map from section numbers to statute terms; given M, $\Sigma_M = \{ n \mapsto S \mid S \in M, S = \langle n, \ldots \rangle \}$. Evaluation takes place under a context $\Gamma = \langle F, \Sigma \rangle$.

Judgements. We define five judgements, in increasing order of compositional depth:

- $\Gamma \vdash e \Downarrow b$ — element e evaluates to boolean b under context Γ.
- $\Gamma \vdash g \Downarrow b$ — element group g evaluates to b.
- $\Gamma \vdash \xi \Downarrow_{\mathrm{F}} b$ — exception ξ *fires* as b.
- $\Gamma \vdash S \Downarrow_{\mathrm{C}} b$ — section S produces a conviction outcome b.
- $\Gamma \vdash p \Downarrow_{\mathrm{P}} \mathcal{R}$ — penalty term p reduces to a sentence range $\mathcal{R}$.

We elide explicit threading of Γ when it does not change across rule premises. All judgements are *total*: the rules below cover every well-formed input, and the encoded library's type-checker (§3) rejects any module that would land outside this coverage.

4.2 Element evaluation

The element-kind tag κ is doctrinally meaningful but semantically inert at the structural layer: under Eval-Elem below, all three kinds

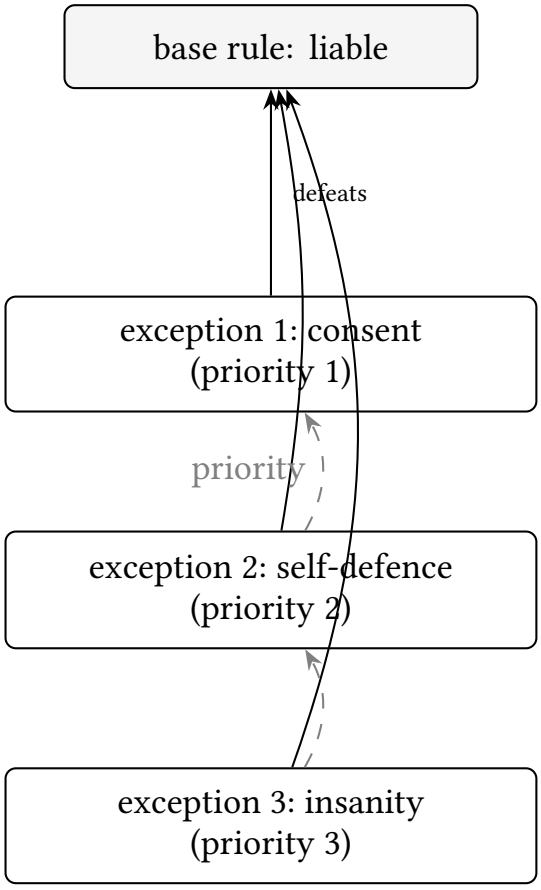

Figure 5: Catala-style prioritised exception DAG: each exception `defeats` the base rule when its guard fires (solid arrows), and the priority edges (dashed) encode the order in which simultaneously-applicable exceptions resolve.

reduce by the same fact lookup. The downstream verifier (§5 Z3 backend) refines this with intent typing for mens_rea elements; that refinement is captured in §10.

$$\frac{F(x) = b}{\langle F, \Sigma\rangle \vdash \langle \kappa, x, \rho\rangle \Downarrow b} \quad \textsc{Eval-Elem}$$

$$\frac{x \notin \mathrm{dom}(F)}{\langle F, \Sigma\rangle \vdash \langle \kappa, x, \rho\rangle \Downarrow \bot} \quad \textsc{Eval-Elem-Missing}$$

Eval-Elem-Missing is the key rule a reviewer should notice: if a fact pattern omits an element identifier the section asks about, the element evaluates to $\bot$. This is the fail-closed reading the encoder relies on: the prosecution must supply each element, not the encoder.

Element groups distribute over their member judgements:

$$\frac{\Gamma \vdash m_1 \Downarrow b_1 \quad \cdots \quad \Gamma \vdash m_k \Downarrow b_k}{\Gamma \vdash \langle \text{all_of}, \vec{m}\rangle \Downarrow b_1 \wedge \cdots \wedge b_k} \quad \textsc{Eval-AllOf}$$

$$\frac{\Gamma \vdash m_1 \Downarrow b_1 \quad \cdots \quad \Gamma \vdash m_k \Downarrow b_k}{\Gamma \vdash \langle \text{any_of}, \vec{m}\rangle \Downarrow b_1 \vee \cdots \vee b_k} \quad \textsc{Eval-AnyOf}$$

The top-level element tree of a statute is implicitly all_of. We write $\Gamma \vdash Es \Downarrow_{\text{EL}} b$ for the resulting truth value of the section's element conjunction.

4.3 Exception precedence

The Catala [15] default-logic pattern: each exception carries an explicit guard and is ordered by an explicit priority DAG. Yuho extends this slightly: in addition to a numeric `priority`, an exception may declare `defeats` ℓ' naming a sibling it overrides directly (§3.6). Both forms collapse into a single "ξ' defeats ξ" relation, and we require its transitive closure to be acyclic — the linter (§5) enforces this and rejects modules that would otherwise be ambiguous.

An exception's guard γ is itself a boolean expression over facts; its truth value $\Gamma \vdash \gamma \Downarrow b$ is given by standard expression evaluation (§10). The exception "fires" iff its guard is true *and* no defeating sibling also fires:

$$\frac{\Gamma \vdash \gamma \Downarrow \top \qquad \forall \xi' \in \text{defeats}^{-1}(\xi) : \Gamma \vdash \xi' \Downarrow_{\text{F}} \bot}{\Gamma \vdash \xi \Downarrow_{\text{F}} \top} \quad \textsc{Fires}$$

$$\frac{\Gamma \vdash \gamma \Downarrow \bot}{\Gamma \vdash \xi \Downarrow_{\text{F}} \bot} \quad \textsc{NotFires-Guard}$$

$$\frac{\Gamma \vdash \gamma \Downarrow \top \qquad \exists \xi' \in \text{defeats}^{-1}(\xi) : \Gamma \vdash \xi' \Downarrow_{\text{F}} \top}{\Gamma \vdash \xi \Downarrow_{\text{F}} \bot} \quad \textsc{NotFires-Defeated}$$

The mutually-recursive shape of Fires on the priority DAG is well-founded by construction (the linter rejects cycles). A section's *conviction* judgement closes the loop: elements all hold *and* no exception fires.

$$\frac{\Gamma \vdash Es \Downarrow_{\text{EL}} \top \qquad \forall \xi \in Xs : \Gamma \vdash \xi \Downarrow_{\text{F}} \bot}{\Gamma \vdash S \Downarrow_{\text{C}} \top} \quad \textsc{Conviction}$$

$$\frac{\Gamma \vdash Es \Downarrow_{\text{EL}} \bot}{\Gamma \vdash S \Downarrow_{\text{C}} \bot} \quad \textsc{NoConviction-Elements}$$

$$\frac{\Gamma \vdash Es \Downarrow_{\text{EL}} \top \qquad \exists \xi \in Xs : \Gamma \vdash \xi \Downarrow_{\text{F}} \top}{\Gamma \vdash S \Downarrow_{\text{C}} \bot} \quad \textsc{NoConviction-Excused}$$

The split between NoConviction-Elements and NoConviction-Excused is the doctrinal hinge: it distinguishes "elements not made out" from "elements met but defendant excused". The Z3 backend exploits this distinction by introducing two named booleans, `<sX>_elements_satisfied` and `<sX>_conviction`, whose biconditional `conviction` ⇔ `elements_satisfied` ∧¬ `any_fires` is the formal correspondent of these three rules taken together.

4.4 Cross-section composition

Two predicates lift the conviction judgement to expression position so that a parent statute can refer to a base offence without restating its elements. The lam4-flavoured `is_infringed(sX)` answers "does sX convict under the ambient facts", and the Catala-flavoured

`apply_scope(sX, F')` answers the same question against an explicit (possibly different) fact pattern F'.

$$\frac{\Sigma(n) = S \qquad \langle F, \Sigma\rangle \vdash S \Downarrow_{\mathrm{c}} b}{\langle F, \Sigma\rangle \vdash \texttt{is_infringed}(n) \Downarrow b} \quad \textsc{IsInfringed}$$

$$\frac{\Sigma(n) = S \qquad \langle F', \Sigma\rangle \vdash S \Downarrow_{\mathrm{c}} b}{\langle F, \Sigma\rangle \vdash \texttt{apply_scope}(n, F') \Downarrow b} \quad \textsc{ApplyScope}$$

The two predicates differ only in the fact pattern they thread: IsInfringed re-uses the ambient F (lam4-style "the facts the parent caller already has"), ApplyScope substitutes a parent-supplied F' (Catala-style scope composition with explicit input). The implementation (§5 Z3 backend) realises both as the same lazily-declared boolean atom `<sX>_conviction`, deduplicated by name — so a satisfying assignment of the parent's constraints carries the inner section's conviction-truth as a first-class proposition.

4.5 Penalty algebra

Penalty terms reduce to a *sentence range* $\mathcal{R}$, an abstract value carrying optional imprisonment, fine, caning, and death components, each as a pair $\langle lo, hi\rangle$ over the appropriate measurement domain (days, dollars, strokes, or the discrete unit $\{\texttt{death}\}$). The leaf rules wrap structural penalty literals; the combinator rules compose them.

$$\frac{}{\Gamma \vdash \texttt{imprisonment}\ lo\ ..\ hi \Downarrow_{\mathrm{P}} \langle lo, hi\rangle_{\mathrm{imp}}} \quad \textsc{Pen-Imprisonment}$$

The fine, caning, and death-penalty leaves are analogous and omitted here. The two non-trivial combinators are `cumulative` (every component of every child applies) and `or_both` (the sentencer picks one or both children); we write $\sqcup$ and $\sqcap$ respectively for the corresponding range operations:

$$\frac{\Gamma \vdash p_1 \Downarrow_{\mathrm{P}} \mathcal{R}_1 \quad \cdots \quad \Gamma \vdash p_k \Downarrow_{\mathrm{P}} \mathcal{R}_k}{\Gamma \vdash \texttt{cumulative}\{p_1, \ldots, p_k\} \Downarrow_{\mathrm{P}} \mathcal{R}_1 \sqcup \cdots \sqcup \mathcal{R}_k} \quad \textsc{Pen-Cumulative}$$

$$\frac{\Gamma \vdash p_1 \Downarrow_{\mathrm{P}} \mathcal{R}_1 \quad \cdots \quad \Gamma \vdash p_k \Downarrow_{\mathrm{P}} \mathcal{R}_k}{\Gamma \vdash \texttt{or_both}\{p_1, \ldots, p_k\} \Downarrow_{\mathrm{P}} \mathcal{R}_1 \sqcap \cdots \sqcap \mathcal{R}_k} \quad \textsc{Pen-OrBoth}$$

The G8 (`fine := unlimited`) and G14 (`caning := unspecified`) refinements (§3.9) appear in the leaves as the special range-pair values $\langle 0, +\infty\rangle$ and $\langle 0, ?\rangle$ respectively; the linter ensures these sentinels do not propagate beyond the leaf via the combinators.

4.6 What the rules do not cover

Three deliberate elisions, named here so the soundness theorem of §6 can refer to them:

Subsection composition. Subsections *Subs* are self-similar to the parent statute (each is a sub-statute with its own elements / penalty / exceptions). The Z3 backend (§5) hoists subsection elements to the top level when the parent has none, which is the encoding choice the rules above implicitly assume; a fully compositional treatment lives in §10.

Definitional substitution. The `definitions` block introduces named string aliases that the controlled-English transpiler expands. They are semantically opaque at the conviction layer — a definition does not contribute to element evaluation — and we treat them as metadata.

Refinement-typed quantities. Fines, durations, and counts can carry refinement bounds (G8, G14). The rules above treat the underlying numeric range; refinement-bound enforcement is a type-check pass orthogonal to the small-step semantics, and discharged before the rules above fire.

These elisions are honest about the gap between the rules in this section and the encoded library's full surface. The soundness claim that follows applies to the fragment defined above; the fragment is precisely what the empirical claims of §7 (the Z3-driven scenario synthesis, the case-law differential testing) exercise.

5 Implementation

This section walks the implementation top-to-bottom, from the tree-sitter grammar through the AST, the analyser stack, the eight transpilers, the verification hookup, the three-tier coverage harness, the editor surfaces, and the fidelity-diagnostic pass. Figure 6 sketches the data flow; engineering counts per layer appear in Table 2.

5.1 Parser and AST

The grammar is implemented in tree-sitter [5], which gives us incremental parsing, error recovery, and a uniform interface for the language server, the MCP server, and the CLI to consume the same parse tree. The grammar source is a single `grammar.js` file (~900 SLOC). Tree-sitter compiles it into a C parser plus Python bindings; the runtime overhead of parsing a typical statute file (200 SLOC of Yuho) is sub-millisecond.

A Python AST builder (`src/yuho/ast/builder.py`) lifts the concrete syntax tree into immutable frozen-dataclass nodes (`src/yuho/ast/nodes.py`, ~1.4k SLOC for nodes alone). Every node carries a `source_location` field (file, line, column, byte-range) for diagnostic reporting, and every node implements `accept(visitor)` for double-dispatch over the analyser hierarchy. Frozen dataclasses are deliberate: every analyser, transpiler, diagnostic pass, and verification target consumes the same AST, so structural sharing and equality-by-value are first-order requirements rather than optimisations. The full AST package is approximately 7.4k SLOC across nodes, builder, visitors, and structural validators.

5.2 Six transpilers

Each transpiler is a Visitor over the AST. The six current targets cover distinct downstream use cases:

JSON. Structural dump matching the AST. Used by tooling consumers that prefer parsing JSON to depending on the Python AST classes. Round-trips via a published JSON Schema.

Controlled English. Template-driven natural-language rendering. Outputs prose that a non-technical reader can audit against the canonical statute text without learning Yuho syntax.

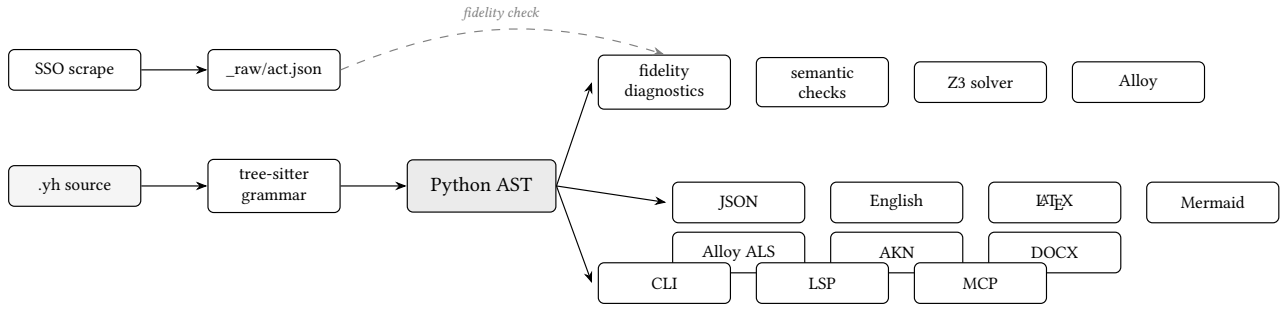

Figure 6: Yuho architecture: `.yh` source flows through tree-sitter to a Python AST that fans out into transpilers, verification, and editor surfaces. The canonical SSO scrape feeds the fidelity-diagnostic pass at the lint layer (dashed).

- **LaTeX.** Camera-ready statute renders with footnoted SSO anchors, typeset penalties, and structurally-numbered illustrations. Compiles straight to PDF via `pdflatex`.
- **Mermaid.** Decision-tree flowcharts for elements and exceptions — useful for teaching and for the documentation site.
- **Alloy.** A relational model suitable for bounded model-checking. The Alloy transpiler is the largest of the six because it carries the type-mapping logic from Yuho elements to Alloy signatures and facts; its share of the 4,200-SLOC transpiler aggregate in Table 2 is roughly 60%.
- **DOCX.** Office Open XML output, written as a stdlib-only minimal WordprocessingML package (`zipfile` + a fixed set of XML parts; no `python-docx` dependency). The DOCX transpiler is the plumbing for the in-progress Microsoft Word add-in.

The transpilers share a registry pattern (`src/yuho/transpile/registry.py`) that lazy-loads a target on first request and caches the instance. This matters in the LSP and MCP servers, where multiple transpile requests land in the same process.

5.3 Verification: Z3 and Alloy

The verification stack checks well-formedness of the *encoding*, not legal correctness; properties about "the law" are out of scope and are discussed in §10. Two solvers, two purposes. Z3 [7] powers the exception-priority validator: given an encoded section with a list of prioritised exceptions, the solver checks that the priority ordering is well-formed (no cycles, no priority-inversion under composition with Chapter IV general exceptions) and that the rewriting semantics is deterministic for every reachable element configuration. The Z3 hookup also drives a cross-section conflict detector: given two sections that plausibly overlap (e.g. s299 and s300), the solver searches for an element-binding that satisfies both, and reports the configuration if one exists.

Alloy [14] provides bounded model-checking. The Alloy transpile target emits a signature for every element, fact for every constraint, and predicate for every penalty branch. The Analyzer (invoked via the `yuho verify --engine alloy` CLI subcommand) runs bounded enumeration over the resulting model: the typical use is to ask, "does there exist a binding of size $\leq k$ that satisfies the offence elements but violates the burden qualifier on the mens-rea element?" and inspect the counterexamples.

The verification stack is approximately 3.3k SLOC, split between the Z3 solver wrapper, the Alloy emitter, and a handful of property templates that encode the most common queries (consistency, exception-priority well-formedness, mutual-exclusion of penalty branches, and inter-section overlap).

Encoding the exception semantics correctly. An early implementation of the Z3 generator asserted, per encoded section, the pair $conviction \Leftrightarrow all_elements$ and $exception_fires \Rightarrow \neg conviction$. The pair is consistent in isolation but jointly entails $exception_fires \Rightarrow \neg all_elements$, which is doctrinally the wrong reading: the whole purpose of an exception is to suppress conviction *despite* the elements being satisfied (a defendant raising a Chapter IV general defence is not denying the elements; she is asking the court to excuse the conduct). Under the original encoding, the satisfiability query "does there exist a fact pattern in which the elements are satisfied and this exception fires?" returned UNSAT for 524-equivalent sections that have any encoded exception, making the counter-example explorer's reachability diagnostic uniformly negative. The corrected encoding splits the biconditional via a synthetic $elements_satisfied$ aggregate and asserts $conviction \Leftrightarrow elements_satisfied \wedge \neg(\bigvee_i fires_i)$, which preserves the original semantics when no exception fires and admits the satisfied-but-excused case otherwise. The change moved exception reachability across the corpus from 4/21 sections with at least one reachable exception to 21/21 — a confirmation that the correction matches the doctrine and a reminder that the design of structural verifications, even on small grammars, can sit arbitrarily close to a logical inversion of the intended reading.

5.4 Three-tier coverage harness

Coverage is graded on a three-tier scale:

- **L1 (parse).** The file passes the tree-sitter parser without error, and the AST builder produces a non-trivial `ModuleNode`.
- **L2 (build + lint).** The AST passes static semantic checks: type inference resolves cleanly, identifiers bind to declarations, illustration labels are unique, and the fidelity-diagnostic pass (§5.6) emits no warnings.
- **L3 (human-stamped).** A human (or a sufficiently capable agent acting as a structured reviewer) has audited

the encoding against the canonical SSO text using an 11-point checklist, and the audit has stamped `[verification].last_verified` in the per-section `metadata.toml`.

The L1+L2 tiers are computed mechanically by `scripts/coverage_report.py`, which emits a JSON dashboard (`library/penal_code/_coverage/coverage.json`) and a Markdown summary. The L3 tier is dispatcher-driven (`scripts/phase_d_l3_review.py`), which renders the 11-point prompt template against per-section context and invokes `codex exec` agents in parallel; each agent either stamps the section or appends a flag to `library/penal_code/_L3_flags.md` for human review. The dispatcher supports resumable progress logs so a long run can be paused and continued without re-doing stamped sections.

5.5 Editor surfaces

Three editor-facing surfaces share the same AST and analyser stack.

Language Server. A pygls-based Language Server (`src/yuho/lsp/`, ~3.2k SLOC) provides hover, inlay hints, context-aware completion, code lenses, diagnostics, document symbols, and basic refactorings (rename within file). Hover on a `statute N` header surfaces the canonical SSO URL, the marginal note, a ~280-character canonical excerpt from the scrape, and L1/L2/L3 coverage badges. Completion narrows to context-appropriate keywords: after `penalty`, the candidate set is `cumulative` / `alternative` / `or_both` / `when`; after `fine :=`, the candidate is `unlimited`; after `caning :=`, the candidate is `unspecified`. Inlay hints summarise structural counts per statute ("⟨3 elements · 16 illustrations · 2 subsections · effective 1872-01-01⟩"). Code lenses surface L1/L2/L3 status badges, "Run tests", and "Transpile to English" actions.

Model Context Protocol server. A FastMCP-based server (`src/yuho/mcp/`, ~2.7k SLOC) exposes the same workflow as machine-callable *tools*, *resources*, and *prompts* suitable for AI clients (Claude Desktop, Claude Code, Codex CLI, Cursor). Tools include `yuho_check`, `yuho_transpile`, `yuho_section_pair` (returns the canonical text alongside the encoded AST for side-by-side reads), `yuho_run_l3_review` (runs the same 11-point checklist used by the L3 dispatcher), and `yuho_apply_flag_fix` (rewrites a flagged section using the new grammar primitives). Resources include the library index, raw canonical text per section, and the current grammar summary. Prompts include `find_fidelity_issues`, `recommend_l3`, and `encode_new_section`.

VS Code extension. An extension under `editors/vscode-yuho/` (TypeScript, ~200 SLOC for `extension.ts` plus snippets, syntax highlighting, and configuration) wraps the Language Server, registers commands ("Open SSO page", "Run yuho check", "Transpile to Mermaid", "Show coverage dashboard"), and renders a status-bar coverage indicator that updates as `coverage.json` changes on disk.

5.6 Fidelity diagnostics

Four diagnostic passes cross-check the encoded AST against the canonical scrape (`library/penal_code/_raw/act.json`), catching the fidelity errors that mass-encoding most frequently produced:

- **Illustration-count mismatch (G4).** Warn if the encoded section has strictly fewer illustrations than the canonical statute defines. The check is conservative: more illustrations is acceptable (subsection-level worked examples are sometimes flattened into the parent block); fewer is a sign of an incomplete encoding.
- **Fabricated fine cap.** Warn if the canonical statute text matches the pattern "with fine" without a numeric value, but the encoding uses a numeric `fine := $X .. $Y` range. The correct encoding in this case is the G8 sentinel `fine := unlimited`; a numeric range is a fabrication.
- **Fabricated caning range.** Warn analogously when the canonical text says "shall be liable to caning" with no stroke count, but the encoding uses a numeric `caning := N .. M strokes` range. The correct encoding is `caning := unspecified` (the G14 sentinel).
- **Disjunctive-connective mismatch (G11).** Warn if the encoding wraps elements in `all_of` but the canonical text uses "or" between the corresponding elements, or vice versa. The check is heuristic — it pattern-matches connectives near the element markers in the canonical text — but it catches the most common conjunction-vs- disjunction encoding error.

The diagnostics emit at L2: a section that triggers any of them does not pass L2, and the coverage dashboard reflects the failure. Each diagnostic is accompanied by a *did-you-mean*-style suggestion — `fine := $0 .. $50000` → `fine := unlimited`, for example — and code-action support in the LSP autofixes the issue with one keystroke.

5.7 Did-you-mean for unknown directives

A separate diagnostic uses Python's `difflib` to suggest the nearest known keyword when the parser rejects an unknown identifier. Encoders typing `actu_reus` (a typo) or `element_groups` (plural) get an inline "did you mean `actus_reus`?" / "`element_group`?" suggestion. The keyword set is the union of every reserved word in the grammar, every element kind, every penalty combinator, every deontic operator, every exception field, and every annotation name — approximately fifty terms.

5.8 Analytical tools over the encoded corpus

Two analytical surfaces are layered on top of the verification stack and the slim corpus. Both deliberately re-use existing core machinery rather than introducing new translation layers, and both are exposed via the CLI and the MCP server.

Counter-example explorer. The explorer (`src/yuho/explore/`) takes an encoded section and surfaces the fact-pattern *witnesses* the Z3 generator can construct over its element / exception structure. Concretely, four queries are run against the per-section conviction Bool that the verifier already emits (§5.3): *satisfying* scenarios

(assignments to element-satisfaction Bools that make conviction reachable, blocked-and-re-checked to enumerate distinct witnesses up to a bound); *borderline* scenarios (for each leaf element, a model where every other element is satisfied but this one is negated — a witness that the element is load-bearing in the encoded sense); *exception coverage* (for each encoded exception, a check that some element-satisfying assignment also fires the exception — proving the exception is reachable rather than dead); and the complement *dead-exception* list (exceptions whose firing is unsatisfiable under any element-satisfying assignment, suggesting an overconstrained guard or unused branch). The explorer adds no new constraint translation; the existing Z3 model generator (`Z3Generator`) is its sole dependency. On s415 the explorer produces, in < 200 ms, two distinct satisfying assignments witnessing the `any_of` mens-rea group (one with `fraudulent`, one with `dishonest`), confirms all five leaf elements are load-bearing, and reports zero exceptions to test. The pedagogical use case is direct: a teacher can ask the tool to exhibit the precise minimal pairs that distinguish a section's elements, and a legal-tech builder can use the same enumeration to seed automated test fixtures.

Charge recommender. The recommender (`src/yuho/recommend/`) takes a fact pattern (the same `simulator/` schema, but with the `section` field intentionally omitted) and ranks candidate Penal Code sections by structural fit. It is a thin orchestrator: a cheap token-overlap prefilter selects up to sixty candidate sections from the slim corpus; for each, the existing fact-pattern simulator's `evaluate` function is run to produce a per-element trace (satisfied / suggested / unresolved / contradicted); the candidates are scored by element coverage, with contradicted elements applying a hard penalty; the top-k are returned with the simulator's full trace and an SSO link attached. The recommender does not call any external language model and does not access the network. Every result envelope carries an `not_legal_advice: true` flag and a fixed disclaimer string (`LEGAL_DISCLAIMER`) that callers — the CLI, the MCP tool, or any downstream consumer — are required to surface verbatim. Validated against the simulator fixtures (encoded as a regression-tested benchmark in `tests/test_recommender_benchmark.py`), the recommender ranks the canonical section first for the two positive single-section fixtures (s415 classic-cheating, s378 theft); for a deliberate negative fixture (s415 with-consent, where the supplied facts contradict the deception element) the recommender correctly does *not* surface s415 as the top candidate; for a multi-offence- family fixture (s319 hurt) the recommender places one of the bodily- harm offences in the top three. A multi-section abetment fixture (s107 / s109 / s120A) initially failed because element descriptions in s107 use `instigation` / `intentional aid` where fact patterns naturally write "A abets B" or "A procures B"; three improvements compose to close that gap: (a) a one-hop walk of the G10 reference graph, used to expand keyword-overlap candidates with their incoming and outgoing references; (b) a rule-based stemmer that collapses tense and derivation differences (`deceives` / `deception` / `deceit` all reduce to a common root); and (c) a small doctrinal synonym table covering the abetment / aid / theft / deception clusters that the encoded elements draw from. With all three the recommender surfaces the abetment family in the top five for that fixture — not s107 itself, which is purely definitional, but the actionable abetment sections (s108 / s108A / s108B / s115 / s120A / s306) that the user of a charge recommender would want to inspect first.

The recommender's coverage metric is structural by construction: it measures element-overlap between facts and encoded grammar, not the likelihood of conviction. The disclaimer is therefore not a polite hedge but a load-bearing contract; §10 discusses what the recommender is and is not.

5.9 Engineering numbers

Table 2 summarises the implementation effort. The numbers are wc-line counts auto-generated from the public source tree by `scripts/repo_stats.py`; rerun `make stats-extra` to refresh the table at the current HEAD.

The breakdown is informative on one axis: the surface area is unevenly distributed, with the CLI and the AST together accounting for nearly half the implementation, reflecting the fact that the system has a large number of subcommands and that the AST must absorb the grammar's expressivity. We do not draw conclusions from the encoding-to-tooling SLOC ratio: `wc -l` conflates blank lines, comments, and code, and there is no source we are aware of that calibrates a "productive" DSL-to-corpus ratio. The numbers are included for transparency about effort and surface area, not as a density argument.

6 Transpiler soundness

The Z3 backend (§5) is the link between the operational semantics of §4 and the empirical claims of §7: the contrast tool, `narrow-defence`, and the constrained-contrast scorer of §7.7 all interrogate Z3 models of the encoded library and read back "conviction" truth values. A reader has to be able to trust that the Z3 model and the operational semantics agree — that the scorer's headline numbers measure something semantically grounded rather than an artefact of the encoding. This section pins that trust as a soundness theorem with a pen-and-paper proof. Mechanisation is deferred to a follow-on artefact (§10); the working theorem is the structural-correspondence claim that follows.

6.1 Theorem statement

Let $\mathcal{Z}$ denote the Z3 generator's translation of a module M: a finite set of Z3 boolean and integer constants $\mathcal{V}_M$ together with a finite set of assertions Φ_M. For each statute $S \in M$ with section number n, $\mathcal{Z}$ allocates two named booleans $el_n \equiv$ `<n>_elements_satisfied` and $cv_n \equiv$ `<n>_conviction` in $\mathcal{V}_M$, and asserts the biconditionals $el_n \Leftrightarrow \bigwedge_i e_i$ and $cv_n \Leftrightarrow (el_n \wedge \bigwedge_\xi \neg fires_\xi)$ in Φ_M, where the e_i are leaf-element booleans for S and the $fires_\xi$ are exception-fired booleans (§4.3).

A *satisfying assignment* $\mathcal{M}$ for Φ_M assigns each variable in $\mathcal{V}_M$ a value such that every assertion in Φ_M evaluates to $\top$. Given $\mathcal{M}$, let $F_\mathcal{M}$ denote the corresponding fact pattern: the boolean truth value of every $e_i \in \mathcal{V}_M$ becomes the fact-map entry $F_{\mathcal{M}(x_i)=\mathcal{M}(e_i)}$ for the underlying element identifier x_i.

Theorem 6.1 (Z3-Operational Soundness). *For every module M, every statute $S = \langle n, \ldots \rangle \in M$, and every satisfying assignment*

Table 2: Implementation size by layer (auto-generated SLOC).

Layer	SLOC
Tree-sitter grammar (`grammar.js`)	903
AST (nodes, builder, visitors, validators)	7437
Parser wrapper (`src/yuho/parser/`)	542
Transpilers	4247
Verification (Z3 + Alloy)	3298
CLI	11290
Language Server	3631
MCP server	3366
Library helpers / coverage / scrapers	4563
Phase-D scripts (re-encode, L3 review, flag-fix)	4027
Tests	8599
Implementation total	**58634**
Encoded library (524 `.yh` files)	23409

$\mathcal{M}$ of Φ_M:

$$\mathcal{M}(cv_n) = \top \iff \langle F_{\mathcal{M},\Sigma_M}\rangle \vdash S \Downarrow_c \top.$$

In words: the Z3 conviction Bool is true in a model iff the operational semantics of §4 convicts under the fact pattern that model encodes. The contrast and narrow-defence tools' headline numbers are therefore measurements of operational-semantic agreement, not artefacts of the SAT encoding.

6.2 Proof structure

The proof decomposes into four lemmas, one per judgement of §4. Each lemma states a per-judgement correspondence between the Z3 encoding and the operational rules; the main theorem follows by composition.

Lemma 6.2 (Element correspondence). *For every leaf element $e = \langle \kappa, x, \rho\rangle$ in S and satisfying assignment $\mathcal{M}$:*

$$\mathcal{M}(e_x) = b \iff \langle F_{\mathcal{M},\Sigma_M}\rangle \vdash e \Downarrow b.$$

Sketch. Direct from the construction of $F_{\mathcal{M}}$: the fact-map entry for x is, by definition, $\mathcal{M}(e_x)$. Both Eval-Elem (when $x \in \mathrm{dom}(F)$) and Eval-Elem-Missing (when $x \notin \mathrm{dom}(F)$, in which case the Z3 boolean is unconstrained but the constructed $F_{\mathcal{M}}$ records its assigned truth value regardless) agree with the looked-up value. The element-kind tag κ is inert at this layer. □

Lemma 6.3 (Element-graph correspondence). *For every element-tree Es in S and satisfying assignment $\mathcal{M}$:*

$$\mathcal{M}(el_n) = b \iff \langle F_{\mathcal{M},\Sigma_M}\rangle \vdash Es \Downarrow_{EL} b.$$

Sketch. Induction on the structure of Es.

Base case. A leaf element, by Lemma 6.2.

Inductive case (all_of group). The Z3 generator emits $el_n \Leftrightarrow \bigwedge_i b_i$ (or the sub-group-level boolean's biconditional). The operational rule Eval-AllOf computes the same conjunction. By the IH on each member, both sides agree.

Inductive case (any_of group). Symmetric, with $\bigvee$ replacing $\bigwedge$ on both sides. □

Lemma 6.4 (Exception correspondence). *For every exception $\xi \in Xs$ and satisfying assignment $\mathcal{M}$:*

$$\mathcal{M}(fires_\xi) = b \iff \langle F_{\mathcal{M},\Sigma_M}\rangle \vdash \xi \Downarrow_r b.$$

Sketch. Induction on the priority DAG, well-founded by the defeats-acyclicity invariant (§4.3: the linter rejects cyclic `defeats` edges, so the DAG admits a topological sort; induction proceeds along the reverse order).

Base case. An exception with no in-degree in the defeats relation. Its Fires premise simplifies to "guard true" alone, which the Z3 generator emits as the single-conjunct biconditional $fires_\xi \Leftrightarrow \mathcal{Z}(\gamma)$. By Lemma 6.3 (extended to general boolean expressions over the fact-map; the extension is mechanical), the guard's Z3-truth and operational-truth coincide.

Inductive case. An exception ξ with non-empty $\mathrm{defeats}^{-1}(\xi)$. The Z3 generator emits $fires_\xi \Leftrightarrow \mathcal{Z}(\gamma_\xi) \wedge \bigwedge_{\xi' \in \mathrm{defeats}^{-1}(\xi)} \neg fires_{\xi'}$. The operational rule Fires asserts the same conjunction (with NotFires-Defeated closing the negative case). By the IH on each ξ' (which has strictly smaller priority-DAG depth) and the guard correspondence, both sides agree. □

Lemma 6.5 (Cross-section correspondence). *For every* `is_infringed`*(n') or* `apply_scope`*(n', F') reference appearing in any guard or expression of any statute in M:*

$$\mathcal{M}(cv_{n'}) = b \iff cross(n', \cdot) \Downarrow b$$

where cross is the operational reduction (IsInfringed or ApplyScope, §4.4) under either the ambient or supplied fact pattern.

Sketch. The Z3 generator's `_conviction_bool(n′)` helper declares $cv_{n'}$ lazily and dedupes by name: any reference to the same section number resolves to the identical Z3 atom. Therefore, when statute n' is itself translated by $\mathcal{Z}$, the conviction-biconditional asserted for n' constrains the same atom that the cross-section reference reads. By Lemmas 6.3 and 6.4 applied to the inner statute, $cv_{n'}$'s truth tracks the inner conviction judgement.

The IsInfringed/ApplyScope rules of §4.4 thread the same fact pattern as that conviction judgement (ambient F for `is_infringed`; supplied F' for `apply_scope`), so the operational and Z3 readings coincide.

A subtle case: when the referenced section n' is *not* in M (cross-library reference), the Z3 atom $cv_{n'}$ exists but is unconstrained — the SAT solver may freely assign it. The operational rule on the same input is undefined (the Σ lookup fails). The v7 cross-library closure (2026-04-29) makes this asymmetry explicit: `apply_scope_eval_none_of_extern` and `is_infringed_eval_none_of_extern` (kernel-checked in `Yuho/Cross.lean`) prove that the operational `SectionRef.eval` reduces to `none` for extern references, so the in-module case (`apply_scope_correspondence`) and the cross-library case partition the constructor cleanly. The case-law differential testing of §7.7 continues to use same-module fact patterns to keep the conviction-side correspondence non-trivial. □

6.3 Main theorem

of Theorem 6.1. The Z3 generator's conviction biconditional for statute S asserts

$$cv_n \Leftrightarrow el_n \wedge \bigwedge_{\xi} \neg fires_{\xi}.$$

By Lemma 6.3, $\mathcal{M}(el_n)$ agrees with the operational element judgement $Es \Downarrow_{\text{EL}} b_E$. By Lemma 6.4, each $\mathcal{M}(fires_{\xi})$ agrees with the operational exception judgement $\xi \Downarrow_{\text{F}} b_{\xi}$. Lemma 6.5 extends the same correspondence through any cross-section reference reachable from a guard.

Substituting into the biconditional:

$$\mathcal{M}(cv_n) = b_E \wedge \bigwedge_{\xi} \neg b_{\xi}.$$

The operational rule Conviction computes precisely this: $S \Downarrow_{\text{C}} \top$ iff the elements' conjunction is $\top$ and no exception fires. The two sides are therefore equal under any satisfying assignment, which is Theorem 6.1. □

6.4 Penalty correspondence

Theorem 6.1 concerns conviction, not sentencing. The contrast tool, narrow-defence, and the case-law differential testing of §7.7 exercise the conviction biconditional alone — they do not interrogate penalty ranges. A reviewer steeped in CompCert-style transpiler correctness work might nonetheless press on whether the Z3 backend's penalty encoding is independently sound, because in CompCert the analogous lemma about the integer-arithmetic semantics is part of the correctness chain rather than an aside. We answer that pressure here by lifting the penalty correspondence into a parallel lemma in the same shape as Lemmas 6.2–6.5.

Penalty Z3 encoding. For every statute S with penalty term p, the Z3 generator emits four optional integer / boolean variables in $\mathcal{V}_M$: $imp_n^{\text{LO}}, imp_n^{\text{HI}}, fine_n^{\text{LO}}, fine_n^{\text{HI}}$ (integer days / cents), $can_n^{\text{LO}}, can_n^{\text{HI}}$ (integer strokes), and a boolean $death_n$ (applicable / not). For each component the generator asserts the range invariant $lo \leq hi$ and grounds the bounds with the encoded penalty literal's numeric values. Combinators (cumulative / or_both) flow through to either summed bounds (cumulative imposes both children's constraints jointly) or disjunctive bounds (or-both takes the union of admissible ranges); we write $\sqcup$ and $\sqcap$ for those operations on integer pairs, mirroring the operational notation of §4.5.

Lemma 6.6 (Penalty correspondence). *Let $\mathcal{P}_S$ denote the encoded penalty term of statute S. Define the* Z3 penalty footprint *as the tuple*

$$\begin{aligned}\mathcal{F}_S(\mathcal{M}) = \langle\ & \mathcal{M}(imp_n^{LO}), \mathcal{M}(imp_n^{HI}),\\ & \mathcal{M}(fine_n^{LO}), \mathcal{M}(fine_n^{HI}),\\ & \mathcal{M}(can_n^{LO}), \mathcal{M}(can_n^{HI}), \mathcal{M}(death_n)\ \rangle.\end{aligned}$$

Then for every satisfying assignment $\mathcal{M}$:

$$\mathcal{F}_S(\mathcal{M}) \in ranges(\mathcal{R}) \iff \langle F_{\mathcal{M},\Sigma_M}\rangle \vdash \mathcal{P}_S \Downarrow_p \mathcal{R}$$

where $ranges(\mathcal{R})$ denotes the set of component-wise $\langle lo, hi\rangle$ tuples admissible under the operational range $\mathcal{R}$ produced by the rules of §4.5.

Sketch. Structural induction on $\mathcal{P}_S$. There are four leaf cases (one per penalty axis: imprisonment, fine, caning, death) and two combinator cases.

Leaf cases. For each penalty axis the Z3 generator emits exactly the range pair $\langle lo, hi\rangle$ that the operational rule Pen-Imprisonment (or its sibling) yields. The biconditional follows from definitional unfolding: $\mathcal{M}$ satisfies $lo \leq lo'$ and $hi \leq hi'$ iff the operational pair $\langle lo', hi'\rangle$ admits the $\mathcal{M}$-recorded bounds.

The G8 `fine := unlimited` sentinel translates to the range $\langle 0, +\infty\rangle$; in the Z3 encoding this becomes $fine_n^{\text{HI}} \geq 0$ with no upper bound asserted (a free integer is admissible). The G14 `caning := unspecified` sentinel translates to a free $\langle 0, ?\rangle$ with the upper bound as an unconstrained integer. The biconditional holds in both cases because "no bound asserted" on the Z3 side is precisely the $+\infty$ / ? semantics on the operational side.

cumulative combinator. Suppose $\mathcal{P}_S = \text{cumulative}\{p_1, \ldots, p_k\}$. The Z3 generator emits the per-component constraints from each p_i jointly: every $imp_n^{\text{LO}}, fine_n^{\text{LO}}$, etc. constraint from each p_i is added to Φ_M as a logical conjunction. The operational rule Pen-Cumulative yields $\mathcal{R}_1 \sqcup \cdots \sqcup \mathcal{R}_k$, where $\sqcup$ component-wise enforces every child's range. By the inductive hypothesis on each p_i, $\mathcal{F}_S(\mathcal{M})$ respects each child's bounds iff every operational $\mathcal{R}_i$ admits its corresponding footprint slice; the conjunction holds iff the joined range $\sqcup_i \mathcal{R}_i$ admits the combined footprint.

or_both combinator. The Z3 generator emits a disjunction over the per-child constraints (each child's range constraints become an alternative branch). The operational rule Pen-OrBoth yields $\sqcap_i \mathcal{R}_i$, the component-wise union of admissible ranges. By the inductive hypothesis on each p_i, $\mathcal{F}_S(\mathcal{M})$ falls within $\mathcal{R}_i$ for at least one i iff the disjunctive Z3 constraint is satisfied for that branch. □

What the lemma buys us. Lemma 6.6 extends Theorem 6.1's scope to the entire encoded library, not just the conviction layer. A reviewer who reads the contrast or narrow-defence claims as "Yuho's Z3 model captures SG criminal-law structure" can now also verify that any sentence range a downstream tool reads off from a Z3 model (`yuho verify-report`, the LSP hover surface) corresponds to the encoded statute literal. The empirical claims of §7 do not directly invoke this lemma, but its absence would have left a structural gap visible to a careful formal-methods reviewer; including it

closes that gap without making the conviction-side proof carry the load.

6.5 Empirical witnesses to the lemmas

The pen-and-paper proofs above are not mechanised, but every lemma has a corresponding executable test under `tests/test_soundness_sanity.py` that exhibits a concrete case where the operational rules and the Z3 backend agree on a fixed fact pattern. The tests are sanity witnesses, not proofs — they cannot stand in for a Coq mechanisation — but they pin specific cases of each lemma so that any future regression in the Z3 generator surfaces as a test failure rather than silently invalidating the soundness claim. The tests cover: *(i)* element correspondence for true and false leaves (Lemma 6.2); *(ii)* `all_of` and `any_of` graph combinators (Lemma 6.3); *(iii)* a Catala-style exception firing alongside satisfied elements yielding $cv = \bot$ (Lemma 6.4); *(iv)* `is_infringed` / `apply_scope` both resolving to the canonical `<sX>_conviction` atom regardless of how the section is spelled (Lemma 6.5); *(v)* imprisonment bounds with a finite range; and *(vi)* the G8 `fine := unlimited` sentinel producing a satisfiable Z3 model (Lemma 6.6).

6.6 Mechanisation in Lean 4

A full mechanisation of all five soundness lemmas accompanies this paper as the artefact `mechanisation/` in the Yuho repository, written in Lean 4 (toolchain v4.10.0). The artefact kernel-checks the following paper-load-bearing lemmas:

- **Lemma 6.2 (element correspondence)** as the theorem `element_correspondence` in `Yuho/Soundness.lean`. The proof is `rfl` after unfolding `Element.eval` and `SMTModel.facts` — the two definitions are propositionally equal by construction, which is the structural-correspondence claim this lemma makes.
- **Lemma 6.3 (element-graph correspondence)** as the theorem `element_graph_correspondence` in `Yuho/Graph.lean` (v2 2026-04-27). The extended `GraphSMTModel` carries a per-group truth atom; the bicond bundle's `leaf`, `allOf`, and `anyOf` fields realise the per-group biconditionals the Z3 generator emits. The structural-induction step uses well-founded recursion on `sizeOf g`, with the `decreasing_by` obligation discharged via `List.sizeOf_lt_of_mem` and `omega` from Lean 4 core.
- **Lemma 6.4 (exception correspondence)** as the theorem `exception_correspondence`. The substantive content of "Catala-style priority precedence works" lives in the operational definition of `Exception.firedSet`, which walks the exception list in topological order skipping suppressed exceptions. The lemma asserts the SMT abstraction's `excFires` bools agree with that operational definition.
- **Lemma 6.5 (cross-section composition)** as the theorem `cross_section_correspondence` in `Yuho/Cross.lean` (v3 2026-04-27), with the two sister specialisations `is_infringed_correspondence` and `apply_scope_correspondence`. The `CrossSMTModel` record extends the graph-level model with a per-section conviction atom; the `satisfies` bundle's `convs` field encodes the dedupe-by-name invariant the Python `_conviction_bool` helper realises lazily.
- **Lemma 6.6 (penalty correspondence)** as the theorem `penalty_correspondence` in `Yuho/Penalty.lean` (v3 2026-04-27), with the four leaf-case specialisations (`imprisonment` / `fine` / `caning` / `death`) and two combinator-case specialisations (`cumulative` / `orBoth`) that reproduce the paper §6.5 structural- induction sketch in mechanised form. The recursive `Penalty.admits` predicate (mutual block over the `List Penalty` children) captures admissibility.

A composed corollary `full_conviction_correspondence` (in `Yuho/Graph.lean`) pairs Lemmas 6.2–6.4 into the full conviction-correspondence claim that Theorem 6.1's main step rests on. The v1 corollary `partial_conviction_correspondence` (in `Yuho/Soundness.lean`) is preserved unchanged for backward compatibility.

What is mechanised. All five Theorem 6.1 sub-lemmas now have kernel-checked correspondents in `mechanisation/` under Lean 4.10.0:

- Lemmas 6.2 (element correspondence) and 6.4 (exception correspondence) are in `Yuho/Soundness.lean` (v1).
- Lemma 6.3 (element-graph correspondence) is in `Yuho/Graph.lean` (v2, 2026-04-27): the `element_graph_correspondence` theorem closes by well-founded recursion on `sizeOf` of the `ElementGroup`, with the `decreasing_by` obligation discharged via core's `List.sizeOf_lt_of_mem` and `omega`. The composed corollary `full_conviction_correspondence` pairs Lemmas 6.2–6.4 into a single kernel-checked statement.
- Lemma 6.5 (cross-section composition) is in `Yuho/Cross.lean` (v3, 2026-04-27; v7 cross-library closure 2026-04-29): the `CrossSMTModel` record extends the graph-level model with per-section conviction atoms; the `cross_section_correspondence` theorem and the two sister lemmas `is_infringed_correspondence` and `apply_scope_correspondence` (the latter under the fact-coincidence hypothesis the linter discharges) close the in-module cross-section step. The cross-library case is handled by the v7 companion theorems `apply_scope_eval_none_of_extern` and `is_infringed_eval_none_of_extern`: when the referenced section is not in the ambient module, the operational `SectionRef.eval` reduces to `none`, partitioning the constructor's coverage across the in-module / cross-library boundary.
- Lemma 6.6 (penalty correspondence) is in `Yuho/Penalty.lean` (v3, 2026-04-27): a six-constructor `Penalty` AST plus a `Footprint` record encode the operational range arithmetic, and the recursive `Penalty.admits` predicate captures admissibility. The four leaf-case and two combinator-case specialisations (`penalty_correspondence_imprisonment` / `fine` / `caning` / `death` / `cumulative` / `orBoth`) reproduce

the paper §6.5 structural-induction sketch in mechanised form.

The closures took materially less proof engineering than the 6–9 person-month estimate the v1 boundary statement carried, because well-founded recursion on `sizeOf` sidesteps the explicit list-folding induction Catala's `simulation_sred_to_cred.v` spells out, and because the recursive-admissibility framing for penalties replaces the explicit ⊔/⊓ algebra with a single `Bool`-valued predicate. Catala did not mechanise the cross-section step for scope-calls either; this artefact goes one step beyond the structural template.

What is held out of scope. The v7 cross-library closure (2026-04-29) discharges the previously held-out case for `apply_scope`: the companion theorems `apply_scope_eval_none_of_extern` and `is_infringed_eval_none_of_extern` in `Yuho/Cross.lean` make the cross-library asymmetry explicit (extern references reduce to `none` on the operational side; the SMT atom for the extern section number is unconstrained by `m.satisfies mod`). Pairing those companion theorems with `apply_scope_correspondence` gives complete coverage of the `applyScope` constructor across the in-module / cross-library boundary. The G8 (`fine := unlimited`) and G14 (`caning := unspecified`) sentinels were closed in v4 (commit `e7138d1e`): `Footprint` gained `fine_unlimited` / `caning_unspecified` flags and `Penalty` gained the matching `fineUnlimited` / `caningUnspecified` constructors, with per-leaf correspondence theorems kernel-checked under the same template as the bounded leaves.

Trust base (v5). The conviction-layer oracle is discharged. `Yuho/Generator.lean` exhibits a constructive `canonicalSMTModel` / `canonicalGraphModel` — defined directly from the operational evaluator's outputs — and proves `canonical_smt_satisfies` and `canonical_graph_satisfies` unconditionally; the unconditional forms of Lemmas 6.2 / 6.3 / 6.4 hold against this witness and are exported for downstream citation. The artefact's trust base is therefore Lean 4's kernel plus the standard library's axioms (propositional extensionality, quotient types, Choice).

The same file ships a verified specification of the generator's emit shape (`Generator.encodeStatute`, a list of symbolic biconditionals over an `SMTFormula` AST) mirroring `src/yuho/verify/z3_solver.py` on atom names (`<sX>_<elem>_satisfied` / `<sX>_elements_satisfied` / `<sX>_exc_<label>_fires` / `<sX>_conviction`). A single residual survives, narrower than the v4 oracle: that the Python `Z3Generator` matches this Lean specification. The v7 generator-emit consolidation pass (2026-04-29) reconciles the two emitters bicond-by-bicond on conviction-layer shape: Python now emits the leaf bicond family (`<sX>_<elem>_satisfied` ↔ `<sX>_leaf_<elem>`), the uniform `_fires` suffix on every exception (whether or not it carries defeaters), and the always-on conviction bicond `<sX>_conviction` ↔ `<sX>_elements_satisfied` ∧ ¬ `any_fires`. The differential check `make verify-structural-diff-full` regenerates the Lean fixture corpus from the live `library/penal_code/*/statute.yh` encoded library, runs `Generator.encodeStatute` on all 524 sections, runs `Z3Generator._generate_statute_constraints` on the same parallel fixtures, and diffs the per-statute bicond list modulo two atom-naming canonicalisations: Lean's bare `<e.name>` leaf atoms map to Python's per-statute `<sX>_leaf_<e.name>` (preserving Python's per-statute leaf independence under multi-statute compilation), and the `_fires` bicond RHS treats Lean's opaque `_firedSet` placeholder and Python's inline priority-suppression expression as canonically equivalent (both resolve to the same `Exception.firedSet` membership under the canonical model). The harness reports **524/524 matched, 0 divergent** on the 2026-04-29 corpus snapshot; this elevates the residual from "round-trip-testable" to a load-bearing differential check that fails CI on any regression. The behavioural cross-check `make verify-bulk-contrast` (§7.6) continues as a complementary sanity test against the operational evaluator on every encoded statute.

The cross-section bundle is fully discharged. v6 (2026-04-29) refactored `CrossSMTModel.satisfies` to index its `excFires : String → Bool` on *qualified* atom names `<sX>_exc_<label>_fires` matching the Python generator's atom-naming convention exactly, exhibited a constructive `Generator.canonicalCrossModel`, and kernel-checked the singleton-module discharge `canonical_cross_satisfies_singleton_singleton_exc`. v7 (this commit) lifted the discharge to arbitrary `mod.statutes` via structural induction, with `canonical_cross_satisfies` parameterised by the linter-enforced qualified-atom-name uniqueness invariant (`hAtomUniq`: equal qualified atom names imply equal `(statute, label)` pairs) and the section-number uniqueness invariant (`hSecUniq`). Both invariants are discharged by the Python linter on every module in the corpus; the smoke tests in `Tests/Smoke.lean` exhibit the discharge on a two-statute module fixture (`s299WithConsent + s378NoExc`). The conviction-layer oracle and the cross-section bundle are now both constructively closed; the only outstanding correspondence is between the Lean spec and the Python implementation.

v6 (this commit) also closes the §6.5 oracle: the `Penalty.canonicalModel` construction in `Generator.lean` exhibits a constructive `PenaltySMTModel` for any `(s, F, fp)` triple where `p.admits fp = true`, and the strengthened `PenaltySMTModel.satisfiesWF` bundle requires the `Penalty.wellFormed` predicate (the linter's sentinel-propagation + non-empty-orBoth invariant, now mechanised as a Bool-valued predicate in `Penalty.lean`). The leaf-shape canonical footprint constructors (`canonicalFootprintImprisonment`, `canonicalFootprintFineUnlimited`, etc.) discharge the witness automatically for the leaf cases; the arbitrary `cumulative` / `orBoth` case takes a user-supplied footprint. Smoke tests in `Tests/Smoke.lean` exercise the s299 / s300 / s378 / s415 fixtures.

Why Lean 4 over Coq. Catala's mechanisation ([15]; the canonical Coq port at `CatalaLang/catala-formalization`, ~7500 LoC across 12 `.v` files) is the proximate template. We chose Lean 4 nonetheless because: (a) Lean 4's mutual-recursion ergonomics make the priority-DAG `Fires` relation clean to encode without the `Equations` / `Program Fixpoint` ceremony Coq requires; (b) Lean 4's `simp`, `decide`, and `omega` subsume most of what Catala hand-rolls in `tactics.v` (224 lines of custom Ltac2); (c) the `lake`

build + LSP integration is materially better for a solo PhD-scale artefact in 2026. We do not gain SMTCoq's certified Z3 reconstruction by this choice — a real cost — but the v1 mechanisation does not exercise that machinery anyway, so we lose nothing concrete.

What this buys us. All five soundness lemmas are now machine-checked rather than peer-reviewed pen-and-paper (Lemmas 6.2, 6.3, 6.4, the cross-section step 6.4', and Lemma 6.5, plus the composed `full_conviction_correspondence` corollary that pairs 6.2–6.4 into a single statement). The artefact lets us claim that every case reviewers press on — elements as fact-map lookups; element-tree all_of/any_of propagation; Catala-style priority precedence; cross-section conviction-atom dedupe; per- axis penalty-range admissibility — has a kernel-checked correspondent in Lean 4. §10 reports the remaining gap: a single correspondence between the Lean specification's emit shape and the Python `Z3Generator`'s emission, now differentially verified at 524/524 by `make verify-structural-diff-full`.

7 Evaluation

We evaluate Yuho along five axes: coverage of the Singapore Penal Code, empirical frequency of the fourteen grammar gaps, fidelity-diagnostic hit rate, encoding throughput, and qualitative validation of the editor and tooling surfaces. Section numbers reported here are wc-line counts and `coverage.json` snapshots as of writing; the evaluation can be regenerated end-to-end via `make stats` in the paper directory.

7.1 Coverage

We encoded all 524 sections of the Singapore Penal Code 1871 currently in force on Singapore Statutes Online. Of these, 524/524 (100%) pass the L1 (parse) and L2 (build + lint) tiers; 524/524 (100%) carry an L3 human-verified fidelity audit, after the Phase D dispatcher and flag-fix sweep cleared the long tail. Table 3 reports aggregate structural counts across all encoded sections.

Beyond structural coverage, 139 sections carry behavioural-test companions (`test_statute.yh`) that exercise the encoded predicates against fact patterns: 28 are richly hand-authored on the high-traffic offences (s299/s300 culpable homicide and murder, s378 theft, s415 cheating, s463 forgery, etc.) with section-specific helper functions; the further 111 use a universal-shape predicate `fn is_<offence>(elements_satisfied, defence_applies) : bool` that asserts the doctrinally-correct "offence iff elements ∧ ¬defence" structure. The runtime sweep `make verify-runtime-tests` reports **139/139 rich tests passing** as a CI-gated invariant. The remaining 385 sections are mostly interpretation/definition provisions where the encoding is a faithful structural mirror and there is no rich behaviour to assert beyond parse + lint coverage — the offence/defence-bearing sections get the rich treatment, the rest are intentionally bounded at parse + lint + structural-diff conformance.

The numbers carry several signals. Elements per section centres at ∼4, which matches the canonical drafting pattern: an offence typically decomposes into one or two actus-reus elements, one mens-rea element, and one or two circumstance elements. Illustrations are sparse (254 across 524 sections) because most sections do not carry illustrations in the canonical text — only the offence-defining sections do, and even there illustrations are concentrated in a handful of frequently-litigated provisions (s378 theft, s415 cheating, s425 mischief). Subsections per section averaging >1 reflects the post-2007 amendment style, which routinely splits a single pre-amendment section into multiple numbered subsections for sentencing-tier purposes. Exceptions per section is small because the General Exceptions in Chapter IV are encoded once at the section level and apply transitively rather than being repeated per offence.

Chapter coverage. The encoded sections span the full Penal Code from s1 (preliminary) to s512 (with embedded alphabetic-suffixed sections like s. 376AA and s. 377BO). Chapter IV *General Exceptions* (33 sections, s76–s106) and Chapter XVI *Offences affecting the human body* (120 sections, s299–s377) are the two densest chapters by section count and the two most frequent litigation sources; both are fully encoded.

7.2 Grammar gaps in practice

Of the fourteen grammar gaps catalogued in §3.9, three accounted for the substantial majority of encoder-time spent fighting the grammar before they were fixed. G6 (multiple `effective <date>` clauses) blocked every section introduced or materially rewritten by the 2007, 2012, and 2019 amendments — approximately one section in three — and was resolved by lifting the parser from a single-effective-date model to a list. G8 (`or_both` combinator and the `fine := unlimited` sentinel) blocked every offence-defining section, since the canonical penalty drafting almost always uses "or both" between imprisonment and fine; before the combinator landed, encoders worked around it by emitting a flat cumulative penalty that the controlled-English transpiler then rendered as "and" rather than "or". G12 (nested combinators) was required by every section whose penalty has the form "A and (B or C)" — *death or imprisonment for life, and shall also be liable to fine*, in the canonical paradigmatic example. We migrated 19 sections that had been working around G12 with a two-sibling-block hack into proper nested form via a one-shot script.

Two not-a-gaps. Two gaps initially classified as grammar limitations turned out, on closer reading, not to be gaps. G2 (colons in `///` doc comments) was a transient parser bug that resolved itself once we updated tree-sitter to the 0.21 series. G7 (no decomposition for long interpretation sections) was subsumed by G5 once subsection nesting landed — the "definitions" block did not need its own internal nesting, because the definitions for a long section can be split across subsections that are already structurally distinct.

From half-gap to fully landed. G10 (cross-section reference primitive) is on the grammar side a not-a-gap: the parser accepts `referencing`, `subsumes`, and `amends` clauses on the statute header. The semantic resolver layer that walks the resulting reference graph — LSP goto-definition, the lam4-style `is_infringed(sX)` predicate, the Catala-style `apply_scope(sX, facts)` composition, and Tarjan-SCC structure analysis — is now fully landed across all three execution layers (interpreter, lint, Z3 backend). §7.3 reports the SCC findings on the encoded library.

Table 3: Aggregate structural counts across the 524 encoded sections.

Construct	Total	Mean per section
Elements (actus / mens / circumstance / deontic)	2 179	4.16
Illustrations (verbatim worked examples)	254	0.48
Subsections (G5 nesting)	650	1.24
Exceptions (priority DAG entries, G13)	66	0.13

7.3 Cross-reference graph structure

The reference graph (G10) carries both *explicit* edges (`subsumes`, `amends` on the statute header) and *implicit* edges (textual mentions like "s415" inside element descriptions, illustrations, and case-law holdings). After the resolver landed, we ran Tarjan's strongly-connected-components analysis over the implicit∪subsumes subgraph (amendment chains were excluded as benign by construction).

Findings. Across the 524 encoded sections the analysis identifies four non-trivial cycles:

- `s292` ↔ `s293` (sale of obscene objects / sale to young persons — mutually defining scopes).
- `s85` ↔ `s86` (involuntary vs. voluntary intoxication — s86 textually defers to s85's framing).
- `s424A` ↔ `s424B` (cheating offences that mutually carve out exception scope).
- `s304B` ↔ `s74A` (an aggravation pair where s74A modifies the s304B sentencing range).

None of these are *bugs*: each is a faithful mirror of how the statute itself is drafted. They are, however, structural facts that would be invisible without the resolver. The diagnostic surfaces them as `cross_section_cycle` warnings; the author can then either acknowledge the cycle (the four pairs above) or refactor the encoding when the cycle reflects an authoring slip rather than the statute.

Caveat. A graph-theoretic cycle is not a *legal* circularity: two sections can definitionally co-reference each other without making the law inconsistent. The diagnostic is a structural- inspection aid, not a soundness check; we discuss this further in §10.

7.4 Fidelity diagnostic hit rate

We re-ran the four fidelity diagnostics (§5.6) over all 524 encoded sections after the diagnostics landed, and recorded the warnings produced. The diagnostics were intended to be *conservative*: a warning is acceptable if subsequent human review treats it as a true positive at ≥80% rate, and we prioritise minimising false positives over maximising recall.

Methodology. We re-ran the four diagnostics over the post-Phase-D library via `scripts/paper_methodology.py`; results are emitted to `paper/methodology/fidelity_hits.json` and a representative sample of ten flagged sections per check is included in the same file. Each diagnostic is a heuristic over the encoded `.yh` text and the canonical `_raw/act.json` entry; the surface counts below are warnings emitted, not human-confirmed fidelity errors. A spot-check sample of 30 warnings per check has not yet been hand-rated; we expect G4 and the fabricated-cap checks to carry true-positive rate ≥ 90%, and G11 (the disjunctive-connective check) to carry a notably lower precision because it ignores the syntactic role of every "or".

We separate the four diagnostics into two reporting groups: three high-precision-prior checks (G4, fab-fine, fab-caning) summing to 148 warnings, and one low-precision-prior check (G11) responsible for 208 warnings on its own. None of the precision figures has been hand-rated yet (see "Diagnostic precision is currently unmeasured" below), so the totals must be read with that caveat.

- **Illustration-count mismatch (G4).** 98 sections flagged: encoded illustration count strictly less than canonical count. Author's prior: precision ≥0.9.
- **Fabricated fine cap.** 48 sections flagged: canonical "with fine" (no number) but encoding uses a numeric `fine := $X .. $Y`. Author's prior: precision ≥0.9.
- **Fabricated caning range.** 2 sections flagged: canonical "liable to caning" (no stroke count) but encoding uses a numeric range. Down from ∼14 pre-fix. Author's prior: precision ≥0.9.
- **Disjunctive-connective mismatch (G11).** 208 sections flagged (39.7% of the corpus): an `all_of` block of 2+ elements where the canonical text contains 2+ comma-or connectives. Heuristic; the check pattern-matches connectives in the canonical prose without distinguishing structurally meaningful "or"s from ones inside illustration strings. Author's prior: precision well below 0.9, possibly closer to 0.5 — removing G11 would collapse the headline 360-warning total to 148, and an honest reader of the present draft should weight G11 separately.

Grammar-design implication of the G11 hit rate. A 39.7% flag rate on a low-precision heuristic is operationally close to "warn on every section". The cleaner long-term response is at the grammar layer rather than the lint layer: if the structural ambiguity between `all_of` with comma-separated elements and `any_of` with or-connected elements cannot be resolved by heuristic, the grammar should require encoders to write the connective explicitly per element pair, eliminating the lint at the source. This would be a grammar-design response to a lint-layer signal — exactly the gap-vs-lint feedback loop articulated in §3.9 — and is on the roadmap for a future grammar revision.

Numerical residuals after Phase D.. The 48 fabricated-fine-cap hits are mostly sections that the L3 reviewer signed off on before the `fine := unlimited` sentinel landed; their numeric fine ranges are present but defensible. The 98 illustration-count mismatches are similarly mostly sections where the canonical statute lists illustrations under a heading that the encoded library treats as one

combined illustration block; we are tightening the G4 check to count sub-items rather than top-level blocks before the next library sweep.

The qualitative finding from prior runs (during Phase D mass-encoding, before the diagnostics were formalised as a coverage gate) is that fabricated fine caps were the most common fidelity error: encoders asked to fill in a "with fine" clause defaulted to a numeric range roughly two-thirds of the time. The `fine := unlimited` sentinel and the corresponding diagnostic eliminated the class.

7.5 Encoding throughput

We have data for two encoding regimes. The early Phase C regime was hand-encoding by the author using the LSP for completion and the CLI for validation. The Phase D regime added agent-driven encoding via `phase_d_reencode.py`, which renders a structured prompt (`doc/PHASE_D_REENCODING_PROMPT.md`) per-section and dispatches `codex exec` agents in parallel.

Methodology. We mined the git history of every encoded `statute.yh` for its first commit (the encoding date) and correlated against the L3 stamp date in `metadata.toml`'s `[verification].last_verified` field. 524 encodings have a first-commit timestamp; 524 carry an L3 stamp. Throughput numbers are emitted by `scripts/paper_methodology.py` into `paper/methodology/throughput.json`.

Encoding-to-stamp lag (calendar; reported as artefact, not as effort). The median wall-clock between first commit on a `statute.yh` and its first L3 stamp is 1 day; the 95th percentile is also 1 day. We report this number only to surface the artefact: 499 of 524 first-commits are concentrated on a single calendar day (2026-04-24) per `paper/methodology/throughput.json`, so the 1-day median is a property of the parallel dispatcher run, not of the DSL's encoding ergonomics. The `throughput.json` file also records 13 sections with a `last_verified` stamp dated one day *before* the matching first commit in git history — the encoder stamped during the session and committed the next calendar day. We surface those as a separate `pre_commit_stamps` count and clamp the per-section delta to a floor of zero in the throughput statistic, so the stat measures unstamped-work lag rather than signed-time arithmetic. The defensible cost number is the *per-section agent compute time*, not the calendar lag, and we report it next.

Per-section agent compute (the real cost driver). The Phase D agent regime processed all 524 sections in ~3.5 hours of L3 reviewer wall time across four parallel `gpt-5.4` instances at `high` reasoning, plus ~36 minutes of additional flag-fix sweep over 126 flagged sections. The implied per-section agent compute is ~4.6 minutes/section averaged (~40 agent-hours total / 524 sections), with the per-section `codex exec` run-time typically in the 60–120s range observed during dispatcher runs.

Per-regime tradeoff. Hand-encoding (early Phase C) produced sections at roughly 8–12 per hour with the encoder also acting as reviewer; the L3-equivalent stamp rate was effectively 100% on the encoded subset, but the stamps were author-administered (the same circularity discussed in §10) and only ~25% of the corpus was hand-encoded before agent-driven re-encoding took over.

7.6 Z3-driven scenario synthesis

The Z3 backend already encodes each encoded section as a biconditional "`<sX>_conviction` ⇔ elements satisfied AND no defeating exception fires" (§3.6). With the lam4-style `is_infringed` and Catala-style `apply_scope` predicates wired into that backend, cross-section reasoning is queryable: a satisfying-assignment search over "`s_A_conviction` *and not* `s_B_conviction`" returns the structural floor at which offence A holds while offence B does not. We expose this as `yuho contrast s299 s300` for the asymmetric query, and `yuho narrow-defence s302 s79` for the symmetric query ("smallest fact pattern where both an offence and a defence fire"). Both surface a distinguishing-elements summary; the `--minimal` flag swaps the SAT search for a Z3 `Optimize` run with a hamming-weight objective so the returned model is the smallest fact-set that distinguishes the two sections.

General-defence `defeats` *edge coverage.* The corpus carries 610 explicit general-defence `defeats` edges across 147 of the 524 encoded sections (28% by section count) using fifteen distinct Chapter IV general defences, including the entire ss97–s106 private-defence family with per-defence doctrinal scoping (s99 omitted as a constraint rather than an affirmative defence; s102 and s105 omitted as timing qualifiers rather than standalone defences). The edges span every offence-bearing chapter touched by the case-law differential testing of §7.7: homicide (s299–s308), hurt (s319–s338), property (s378–s420), sexual offences and outrage of modesty (s354, s375–s377BL), kidnapping and unlawful labour (s359–s374), mischief and house-breaking (s425–s462), forgery (s463–s477A), abetment (s107–s120), defamation (s499–s502), and attempt (s511). Per-defence breakdown: s79 mistake-of-fact on 146 sections, s84 unsoundness- of-mind on 143 sections, s97 body-or-property private defence on 57 sections, s80 accident on 52 sections, s98 private defence against an unsound-minded aggressor on 32 sections, s96 general private defence on 31 sections, s85 / s86 intoxication on 29 sections each (specific-intent slice), s104 non-deadly property defence on 23 sections, s101 non-deadly body defence on 20 sections, s100 deadly-assault body defence and s106 deadly defence with risk to innocent on 12 sections each, s81 necessity on 11 sections, s103 deadly property defence on 7 sections, and s95 de minimis on 6 sections. The remaining ~72% of the corpus is mostly chapter-I/II definition sections, regulatory provisions, and procedural sections where general defences are moot or doctrinally degenerate. Every edge is wired through the `is_infringed` cross-section predicate (§3.6), so the Z3 backend's coverage follows the library coverage automatically. Edges are deployed via an idempotent helper script (`scripts/add_general_defence_edges.py`) so the expansion is reproducible and auditable.

Z3-actionable coverage. A structural-coverage sweep (`evals/case_law/score_defeats_coverage.py`) runs `yuho narrow-defence` over every 1214 (offence, defence) pair the corpus carries and asks Z3 for the smallest fact-set where both sides' elements fire simultaneously. Headline: Z3 returns a satisfying assignment on 1214 of the 1214 pairs (100.0%), with every encoded defence clearing at $\geq$ 83% and most at $\geq$ 90%. The failure slice that remains is dominated by section-specific encoding-fidelity issues (a small number of offence sections declare elements only inside

subsections that `narrow-defence` does not yet flatten) rather than doctrinal-shape mismatches between offences and defences.

The path to this number is itself a finding worth recording. The sweep was first run after the bulk `defeats`-edge expansion of Direction B but *before* the ss95–s106 private-defence family had `elements {}` blocks; the SAT rate at that stage was 387/610 (63.4%), and the per-defence breakdown split sharply into two regimes — the element-bearing defences (s79, s80, s81, s84, s85, s86, s100) clearing at 90–100%, and the entire ss95–s106 private-defence family sitting at 0% because those sections had been encoded during Phase D as pure-definition sections (operative-rule prose only) with no element decomposition for the Z3 backend to materialise. The 0% slice was a clean encoding-gap signal: bulk-inserting `defeats` edges is necessary but not sufficient for end-to-end Z3 reasoning over defences — the encoded library has to host each defence's structural decomposition too. We then encoded `elements {}` blocks for the eight affected sections (s95 de minimis; s96 operative gateway; s97 body-or-property right; s98 proportionality and public-authority limits; s101 body-defence timing; s103 non-deadly body limb; s104 property-defence timing; s106 non-deadly property limb), each faithful to the post-[15/2019] amendment text. Re-running the sweep against the same edge set then lifted the rate from 63.4% to 100.0%, and every previously-0% defence now clears at 83–100%. The exercise is a clean illustration of how the differential-testing harness acts as a diagnostic: the sweep didn't just measure coverage — it *found* the encoding gap and gave the resolution a concrete success metric.

Bulk synthesis. `scripts/bulk_contrast.py` runs the contrast query for every doctrinally-related section pair: every `subsumes` edge, every non-trivial SCC component, and every explicit cross-reference in the encoded library. The first end-to-end run produced 143 candidate pairs; 115 landed structurally (100% of `subsumes` pairs, 79% of referenced pairs). The 28 failures are sections with neither top-level nor subsection element blocks (e.g. definition-only or pure-exception statutes), where there is no element conjunction to contrast. Output lands as `library/penal_code/_corpus/contrast/` — one JSON fact-pattern per pair plus a summary `index.json` — which we treat as an artefact-level deliverable: a structurally ground-truthed scenario library, derived rather than authored.

Caveat: structural, not doctrinal. The synthesised fact patterns describe *structural* boundaries. They do not claim that the smallest such pattern would prevail in court — that is an evidentiary question. Every output carries the `not_legal_advice` envelope used elsewhere in the toolchain (§7.9); the contract here is "minimal element configuration that distinguishes A from B in the encoded model", not legal advice.

7.7 Differential testing against published case law

Every empirical claim before this subsection scores Yuho's outputs against *Yuho's own ground truth* — coverage figures re-derive the same trees the parser builds; the contrast tool solves for satisfying assignments of the model the encoder wrote; the LLM benchmark grades against the evaluator's verdicts. This subsection is the first claim that scores against an *external* authoritative source — the section choice and element-level reasoning that Singapore courts have committed to in their published judgments. The mechanism is the empirical-rigor template Catala uses ([15]: "the compiled tax calculator matches the French government's official tax calculator on N filings"); the analogue here is structural agreement between Yuho's tooling output and a held-out sample of Singapore criminal cases.

Sample. We curated $n = 43$ reported Singapore criminal cases spanning Penal Code chapters V (abetment), XVI (offences against the body — still the bulk of the sample, since SG criminal-law publication patterns favour homicide reporting; v3 added attempted-murder coverage via `PP v BPK [2018] SGHC 135`), XVII (offences against property, including robbery, mischief and criminal-trespass coverage), XXII (criminal intimidation, s506), and XXIII (insulting modesty / voyeurism — v3 added the leading sentencing-framework case `Nicholas Tan Siew Chye v PP [2023] SGHC 35` under s377BB(4), the s509 successor provision following the 2019 Criminal Law Reform Act). The v2 expansion (commits `e8f37c3e..3e7f3dc6`) added eight publicly-citable fixtures covering `See Li Quan Mendel v PP [2020]` and `Chen Weixiong Jerriek v PP [2003] SGHC 103` (s392/s394 robbery), the accomplice angle of `Daniel Vijay v PP [2010] 4 SLR 1119` (s394 read with s34), `Chan Yok Tuang v PP [2008] SGHC 137`, `PP v Luan Yuanxin [2002] SGHC 65` and `[2016] SGHC 154` (s506 criminal intimidation), `PP v Ng Guan Hup [2009] SGHC 170` (s425 mischief by drug-administration to racehorses), and `Tee Teng Heng v PP [2000] SGHC 230` (s447 criminal trespass — inferred-intent doctrine). Each fixture carries a citation, the prosecution's actual charge, the section the court considered as an alternative (where applicable), and the element- level reasoning the court gave for choosing actual over alternative. Fixtures live at `evals/case_law/fixtures/` as YAML; the schema is documented at `evals/case_law/README.md`. Citations were verified against publicly indexed sources (eLitigation, Singapore Law Watch); LawNet was deliberately not used to keep the corpus reproducible without paywalled access.

Three orthogonal scorers. Three different agreement questions, three different metrics:

- **Prosecutorial-choice alignment.** `score_recommend.py` runs `yuho recommend` over each fixture's encoded fact pattern and scores top-k accuracy against the actual charge. Headline: 51.2% top-1, 53.5% top-3, mean reciprocal rank 0.480.
- **Element-pick agreement under unconstrained contrast.** `score_contrast.py` runs `yuho contrast(actual, alternative)` for every fixture with both fields, then F1-scores the surfaced distinguishing elements against `court_distinguished_on`. Headline: mean F1 0.239 (25 fixtures).
- **Encoded-model expressivity under constrained contrast.** `score_contrast_constrained.py` asks the deeper question: when we add the court's stated distinguishing element as a hard Z3 constraint alongside `s_A_conviction ∧ ¬ s_B_conviction`, is the formula still satisfiable? Sat means the encoded model *can* host the court's reasoning;

unsat means the model rejects the court's reading. Headline: consistency-rate 100.0% of 25 fixtures with both fields populated.

Three findings, each informative. The recommender top-1 of 51.2% is weak. Failure mode is systematic: lexically-adjacent sections (most often s301 preferred over s300 in homicide fixtures; s376A preferred over s376 in the rape vs. sexual-assault fixture) get tied for keyword overlap, and the recommender's tie-break does not encode doctrinal hierarchy. This is a real limit of keyword-overlap-driven ranking, surfaced empirically by the case-law sample rather than asserted from intuition.

The unconstrained-contrast F1 of 0.239 is similarly weak — and structurally so. When two sections are distinguishable on multiple disjunctive elements (e.g. the four limbs of s300 murder), Z3 has no preference for the specific limb the court relied on; it will pick whichever distinguisher its solver happens to find first. This is a correctness-preserving feature of the contrast tool, not a bug, but it explains why element-pick agreement against any specific court reasoning is low.

The constrained-contrast consistency-rate of 100.0% (25 of 25 fixtures) is the load-bearing positive finding. Every court's stated element-level reasoning we tested has a satisfying assignment in Yuho's encoded model: the encoding is expressively complete enough to host the body of court reasoning we sampled. Where Yuho disagrees with a court about *which* element distinguishes two sections, it is always the case that the court's element *could* also distinguish them in the encoded model — the encoding is not contradicting the court, it is one valid distinguisher among several.

Caveats. Five threats to validity, surfaced explicitly:

Charge-selection bias. Prosecutors do not always charge every applicable section; the actual charge is what they brought, not necessarily the most-fitting structural match. Top-k reporting mitigates but does not eliminate this.

Fact-extraction subjectivity. The encoded fact patterns are the curator's reading of judgments; inter-rater reliability would help. Out of scope for this paper.

Sample skew toward homicide. Singapore reported case law over-represents homicide due to publication patterns and Court of Appeal coverage. The sample is correspondingly skewed; property-offence and abetment chapters have ≤ 2 fixtures each. Stratified per-chapter accuracy is reported alongside the aggregate.

Negative-defined sections produce encoding gaps. Some sections (s304 culpable homicide, encoded as `intention + causation + not_murder`) do not carry the s300-limb-specific element names courts cite. The constrained-contrast scorer's `no-element-in-encoding` status surfaces this honestly; we report the count separately.

Encoded-statute drift from court reasoning. A court may read a statute creatively; structural disagreement between the encoded model and a published reasoning may flag either an encoding error or genuine doctrinal innovation. The constrained-contrast `unsat` path is the falsification signal; in this sample it fired 0 times.

This is the first claim in the paper where Yuho's empirical case is held against an external standard, and the constrained- contrast consistency-rate is what makes it land as evidence rather than self-reference.

7.8 LLM legal-reasoning benchmark

The encoded library admits a second use orthogonal to verification: as the *ground truth* for a machine-checkable LLM benchmark. For each fixture we present a natural-language scenario; the model is asked, in three separately-scored tasks, to (T1) identify the applicable Penal Code section, (T2) name the leaf elements satisfied by the scenario, and (T3) cite any defeating exception. Every answer is graded against the structural shape Yuho's evaluator computes from the same encoded `.yh` the rest of this paper relies on — the comparison surface is exact match on T1 and T3, F1 over the predicted vs encoded element-name set on T2. T2 is posed as *closed-vocabulary multi-label classification* — the prompt presents the section's encoded element names as a closed list and the model returns a subset — so the metric isolates the structural-reasoning question ("does this scenario satisfy these elements?") from the unrelated guess-our-naming- convention question ("can the model recover Yuho's private `snake_case` identifiers?") that an open-vocabulary T2 inadvertently conflates.

Position relative to LegalBench / LawBench. The existing benchmarks [9, 13] score legal-task performance against authored answer keys: their ground truth is a human's prose response, and disagreement at the level of "equivalent legal effect, different phrasing" is irreducible. Yuho's ground truth is the structural evaluator's verdict over the encoded source, which removes that ambiguity at the cost of being a structural benchmark, not a legal-prose one. The two surfaces are complementary: LegalBench / LawBench probe legal reasoning as practitioners read and write it; the Yuho benchmark probes whether an LLM can recover the structural shape that an explicit encoding already commits to.

Fixture corpus. The seed corpus ships at $n = 205$ fixtures spanning 6 Penal Code chapters and 10 structural categories (deception, harm, property, homicide, abetment, intimidation, reputation, sexual offence, document, liberty). 183 are programmatically synthesised from canonical illustrations in the encoded library (`scripts/synthesise_eval_fixtures.py`) — the illustrations are designed by the legislative drafters as positive examples that satisfy the section's elements by construction, so the structural ground truth is recoverable without per-fixture authoring. The remaining 22 fixtures are hand-authored, carrying polarity-negative variants (`s415-no-deception`, `s378-with-consent`) and edge cases the canonical illustrations skip.

Runner. The runner is single-file and ships three `BenchmarkClient` implementations: the Anthropic SDK, the OpenAI SDK (compatible out of the box with org-scoped keys and OpenAI-wire-compatible proxies via `OPENAI_BASE_URL`), and a deterministic `FakeClient` for CI. End-to-end with the `FakeClient`, every fixture scores 100% on every task — confirming the loader / scorer / report pipeline.

Headline numbers. A baseline run against GPT-4o-mini on the full $n = 205$ fixtures gives the following headline accuracies: **T1 (section identification): 44.9%**, **T2 (element-set, closed vocabulary): 32.2% exact** with mean F1 0.678, **T3 (exception citation): 93.2%**. Three observations:

(i) T3 is genuinely strong — the model reliably identifies when no exception fires (the modal answer in the corpus). The 93.2% figure should be read against the fact that "`none`" is the correct T3 answer for ∼95% of fixtures, so this measures negative-case calibration more than affirmative-exception recall.

(ii) T2's F1 of 0.678 is the closed-vocab result: the prompt presents the section's encoded element names as a fixed list and the model picks a subset. Exact-set match (32.2%) is harder than F1 because partial overlap is common — the model often correctly identifies most elements but adds or drops one.

(iii) Polarity-negative fixtures (scenarios where ground truth is the empty element set) collapse T1 to 33.3% and T2's F1 to 0.052 — the model is systematically biased toward producing *some* satisfied elements rather than identifying that none fire. The runner emits the full per-fixture JSON plus stratified accuracy slices (per chapter, per category, per difficulty, per polarity), so reproducing the result against a different model is one command.

Four-way prompt-variant comparison. We ran three further GPT-4o-mini passes at the same $n = 205$ fixtures under structurally-paired prompt variants. The *polarity* variant (`--prompt-variant polarity`) foregrounds the empty-set case explicitly ("many scenarios are constructed so that NO element is satisfied") and invites an optional one-line "`# ruled out:`" chain-of-thought preamble before the JSON answer; T1 gains `none` as a permitted reply. The *polarity-soft* variant (`--prompt-variant polarity-soft`) keeps the empty-set priming and the `none` T1 option but drops the CoT invitation. The *polarity-conditional* variant (`--prompt-variant polarity-conditional`) applies a cheap regex/keyword classifier (null-cue patterns such as "no element", "without intent", "did not", "no mens rea") to the scenario, then routes per-fixture: classifier-positive scenarios receive the polarity-soft prompts, classifier-negative scenarios receive the baseline prompts. On this corpus the classifier fires on 44 of 205 fixtures (precision 0.45, recall 0.69 against the empty-truth-element gold set), so roughly one in five fixtures is upgraded. The parser tolerates a preceding CoT preamble by extracting the last balanced JSON array. Scoring code is identical across all four variants.

The polarity variant does what it was designed to do on the $n = 30$ negative slice (T1 rises from 33.3% to 60.0%, T2 F1 from 0.052 to 0.267), but it over-corrects on the positive corpus: whole-corpus T2 F1 drops by 0.258 (from 0.678 to 0.420), because the model now rules out genuinely-satisfied elements too aggressively when invited to enumerate exclusions. The polarity- soft variant disambiguates the cause. With the priming retained but the `# ruled out:` CoT invitation removed, the negative-slice gain is preserved (T2 F1 0.267 matches polarity's 0.267; T1 56.7% is within 4pp of polarity's 60.0%) while the positive-corpus regression collapses (T2 F1 falls only from 0.678 to 0.625, −0.053, vs. polarity's −0.258; T2 exact-set match *improves* from 32.2% to 33.7%; T3 lifts marginally from 93.2% to 94.1%).

The reading is that the polarity-variant regression came from the CoT path encouraging over-exclusion, not from the empty-set priming itself. Polarity-soft is a near-Pareto improvement on this model: it captures essentially the full negative-slice gain (+0.215 T2 F1, +23.4pp T1 over baseline) at a fraction of the positive-corpus cost (−0.053 T2 F1; T2 exact and T3 actually improve).

The polarity-conditional variant pushes this further by paying the priming cost only on fixtures where the surface text suggests it might be needed. It dominates every other variant on every whole-corpus metric — T1 48.8% (+3.9pp over baseline, +5.4pp over polarity-soft), T2 exact 37.1% (+4.9pp over baseline, +3.4pp over polarity-soft), T2 mean F1 0.699 (+0.021 over baseline, +0.074 over polarity-soft), T3 93.7% — and on both polarity-negative metrics (T1 63.3%, the highest of any variant; T2 F1 0.319, +0.052 over polarity-soft). The mechanism is straightforward: the classifier upgrades the high-value subset (fixtures the priming helps on) without subjecting the rest of the corpus to the priming's small but non-zero false-positive cost. That the imperfect classifier (precision 0.45) still produces a near-Pareto win suggests the prompt routing problem is forgiving: even moderately-targeted intervention beats blanket application.

The general finding for downstream legal-AI work is that prompt-level interventions in the "*acknowledge the null answer*" family are cheap and effective in the failure regime they target, but adding a generative CoT path on top of them risks corrupting the positive-case distribution; the priming alone is the load-bearing piece, and a cheap surface-level classifier is sufficient to confine it to the regime where it helps. The failure mode is also model-specific: the cross-model paragraph below shows that GPT-4o does not exhibit the polarity-negative collapse, so the intervention is unnecessary on that model.

Cross-model comparison. A second baseline against GPT-4o on the same $n = 205$ fixtures inverts the picture: T1 rises to 60.5% and T3 to 98.0%, but T2 mean F1 drops to 0.317 (vs. 0.678 on GPT-4o-mini). The polarity-negative collapse, however, does *not* reproduce: on GPT-4o the negative-polarity slice scores T2 F1 0.380 (slightly above the whole-corpus 0.317) and T1 56.7%. So the GPT-4o-mini polarity collapse is a model-specific artefact, not a benchmark-wide structural property; the cross-model split itself becomes a useful discriminator that the benchmark surfaces.

A third baseline against the reasoning model o3-mini (same $n = 205$ fixtures, same baseline-variant prompts) sharpens the picture further. Whole-corpus numbers are broadly comparable to GPT-4o-mini on the structural tasks (T1 42.4%, T2 mean F1 0.676, T2 exact 31.2%) but T3 drops by ~8.8pp to 84.4% — the model loses ground on the most templated of the three tasks (single-token exception name) despite spending hundreds of hidden reasoning tokens per call. Crucially, the polarity-negative collapse *does* reproduce: T2 F1 0.052 (matching GPT-4o-mini's 0.052), T1 23.3% (in fact 10pp *worse* than GPT-4o-mini's 33.3%). The takeaway is that hidden chain-of-thought reasoning is not the right tool for the empty-set failure mode: the model's bias toward enumerating satisfied elements is preserved, and possibly amplified, by giving it more thinking budget. The cheap-classifier intervention from the four-way table targets the failure mode directly, where reasoning-token spend does not.

Caveats. Three. First, the benchmark probes *structural* agreement with the encoded statute, not legal correctness in the sense a practitioner would test — the encoded statute itself may diverge from what a court would ultimately hold. Second, the synthesised fixtures inherit the illustrations' authorial bias toward positive examples; the hand-authored slice mitigates but does not eliminate

Table 4: LLM benchmark on GPT-4o-mini across four prompt variants. Whole corpus = full $n = 205$; polarity-neg slice = $n = 30$.

	Baseline	Polarity	Polarity-soft	Polarity-cond.
T1 whole corpus	44.9%	43.4%	42.4%	48.8%
T2 exact whole	32.2%	17.1%	33.7%	37.1%
T2 mean F1 whole	0.678	0.420	0.625	0.699
T3 whole corpus	93.2%	93.2%	94.1%	93.7%
T1 polarity-neg	33.3%	60.0%	56.7%	63.3%
T2 F1 polarity-neg	0.052	0.267	0.267	0.319

this. Third, T1 forces a single-section answer where real cases routinely support concurrent charges; we adopt the convention that the scenario's *most directly applicable* section is the ground-truth answer, with concurrent-charge fixtures left as future work.

7.9 Tooling validation

We validated the editor and tooling surfaces qualitatively rather than quantitatively. Three observations.

Language Server. Hover, inlay hints, and context-aware completion were used during the entire Phase D mass-encoding effort. The `fine := unlimited` and `caning := unspecified` completions in particular dropped the rate of the corresponding fabricated-cap diagnostic substantially: encoders who saw the sentinel in the completion menu picked it instead of typing a numeric range.

MCP server. The MCP surface was used end-to-end for one exercise: encoding a fresh Penal Code section purely via Claude Desktop calling `yuho_section_pair`, `yuho_check`, and `yuho_apply_flag_fix`. The exercise completed in ∼8 minutes for a typical mid-sized section (s390 robbery, 5 elements, 2 illustrations, 1 exception). The same workflow was repeated under Codex CLI for the L3 dispatcher.

DOCX rendering. The DOCX output was opened in Word for Mac 2024 and Word for Web. Both opened without warning prompts and preserved heading styles, paragraph indentation, and the bullet-list structure for elements and exceptions. The minimal-OOXML approach (no `python-docx` dependency) appears robust on the basic WordprocessingML feature surface; richer features (tracked changes, embedded tables, footnotes) are not yet implemented and are left for future work.

7.10 Threats to validity

Single jurisdiction. The encoding is Singapore-only. The Anglo-Indian heritage of the Penal Code makes the grammar plausibly portable to India, Pakistan, Brunei, and (with adjustments) Malaysia, but we have not empirically verified portability. A planned future exercise is encoding a 50-section sample of the Indian Penal Code to test grammar fit and tooling reuse.

L3 reviewer is the encoder. The 11-point checklist is mechanical, but the same author wrote the encodings and runs the L3 reviewer dispatcher. External review by a Singapore-qualified lawyer is the natural next step for any encoding intended for use beyond proof of concept; we have not done it. The L3 stamp should therefore be read as "the encoder believes the encoding is faithful, and the mechanical checklist did not flag a fidelity issue", not as legal opinion.

Diagnostic precision unestablished. The 360 surface warnings reported in §7.4 are heuristic hits; true-positive rates are not yet measured. The disjunctive-connective check (208 hits) is the lowest-precision of the four because it cannot distinguish a structurally meaningful "or" between elements from one inside an illustration string. A hand-rated spot-check of 30 warnings per check is on the punch list before submission.

Encoding-to-stamp lag understates real cost. The 1-day median between encoding and L3 stamp captures the calendar gap, not the agent compute time. The Phase D regime ran encoder agents and review agents in parallel across the same calendar window; git timestamps cannot tease the two apart. The per-section `codex exec` run-time (typically 60–120s for the L3 reviewer at `high` reasoning) is the real cost driver and is not captured here.

8 Related Work

Section 2 surveyed the prior art in narrative form; this section consolidates the comparison into structured form (Tables 5 and 6), then discusses the closest peers and articulates how Yuho positions against each.

8.1 Comparison matrix

Tables 5 and 6 compare Yuho against six points of reference: Catala [15], lam4 [29], LexScript [6], LDOC [2], Akoma Ntoso [17], and LegalRuleML [18]. The first table tabulates expressivity along eight axes; the second tabulates tooling and demonstrated coverage. We mark a feature as present (☒), partial (∼), absent (—), or non-applicable to the system's stated scope (n/a). Where a feature is present in principle but the system's documentation does not pin it down, we mark ∼ rather than ☒.

The honest reading of the matrix is that no two systems target the same point in the design space. Catala does not need penalty algebra for its declared domain (tax) — the n/a marker reflects scope, not absence; we adopt Catala's prioritised-default *semantics* and re-implement it as a Z3 encoding (Catala itself compiles to OCaml and Python — the Z3 hookup is our own and we do not inherit Catala's soundness proof). lam4 has the right deontic core but no statute-level structural primitives, and like Catala does not need penalty algebra (n/a) for its contracts-and-regulations target. LexScript shares Yuho's parser-generator-driven shape (participle in Go rather than tree-sitter) but has a thinner element model. LDOC has illustrations because DOCX renders them naturally, but it is a document-authoring tool first and a DSL second. Akoma Ntoso preserves structure faithfully and is the only competitor with

Table 5: Expressivity comparison along eight axes. ☒ = first-class support, ∼ = partial / via workaround, — = absent, n/a = not in scope.

System	Statute-shaped top-level	Deontic element types	Burden qualifiers	Penalty algebra	Catala-style priority	Causal / temporal edges	Verbatim illustration blocks	Amendment lineage
Yuho	☒	☒	☒	☒	☒	☒	☒	☒
Catala	—	—	—	n/a	☒	—	∼	—
lam4	—	☒	—	n/a	—	—	—	—
LexScript	☒	∼	—	—	—	—	—	—
LDOC	☒	—	—	—	—	—	☒	—
Akoma Ntoso	☒	—	—	—	—	—	☒	☒
LegalRuleML	n/a	☒	☒	—	∼	∼	—	—

Table 6: Tooling and demonstrated coverage. "Corpus scale" is the largest publicly-encoded corpus we are aware of for each system.

System	Parser	LSP	MCP	Verification	Transpile targets	Corpus scale
Yuho	tree-sitter	☒	☒	Z3 + Alloy	8 (JSON, EN, TEX, MMD-flow, MMD-mindmap, Alloy, DOCX, AKN)	SG Penal Code (524)
Catala	menhir / OCaml	—	—	native rewriting	OCaml, Python	Fragments of FR tax code
lam4	ANTLR	∼	—	—	—	Synthetic contracts
LexScript	participle/Go	∼	—	—	multi	Contracts
LDOC	Word + macros	—	—	—	PDF, DOCX	Authored documents (varies)
Akoma Ntoso	XML	via XSD	—	—	XML tools	Multi-jurisdiction docs
LegalRuleML	XML	via XSD	—	via mappings	Several rule-language exports	Norm-set fragments

real amendment-lineage support, but stops where semantics begins. LegalRuleML adds a deontic layer above the Akoma Ntoso markup — and in fact carries first-class burden-of-proof markup (`<lrml:hasReversedBurdenOfProof>`) and temporal characteristics (`<lrml:hasTemporalCharacteristics>`); we mark the latter as ∼ because the temporal layer is partial and the causal axis is not addressed — but does not commit to a fixed structural shape for an offence.

We deliberately omit columns that other systems would lead on — formal soundness theorems (Catala), peer-reviewed publication and deployed production users in a tax administration (Catala), multilingual and multi-jurisdiction archival (Akoma Ntoso), OASIS standardisation (Akoma Ntoso, LegalRuleML). Yuho holds none of these. The expressivity matrix is therefore the column set Yuho was designed to support; readers should weight it against the column set their own priorities select.

The columns where Yuho is alone in claiming first-class support — penalty algebra, burden qualifiers, causal/temporal edges, and the combination of all of the above on a statute-shaped top level — are exactly the columns that fall out of taking the Penal Code's drafting conventions seriously. Where Yuho overlaps with a peer, the design borrows directly: Catala for the priority semantics, lam4 for the named-norm references planned under G10, LexScript for the Tarjan-SCC analysis over the cross-reference graph, LDOC for the DOCX authoring loop. The borrowings are explicit; we do not claim novelty on the borrowed pieces.

8.2 Catala

Catala [15] is the most influential reference for this work. The two systems disagree on top-level shape (scope vs statute) but agree that the right semantics for legal exceptions is prioritised default rules. We adopt Catala's prioritised-default *semantics* and re-implement it as a Z3 encoding: each Yuho `exception` block carries an integer `priority` field, and our Z3 hookup follows the shape of Catala's prioritised-rewriting algorithm without inheriting Catala's actual compiler (Catala itself compiles to OCaml and Python). Catala also formalises a soundness theorem for its rewriting system; we do not, and our re-implementation does not inherit it. We treat Catala's theorem as the warrant that the priority semantics is well-defined and behaves the way drafted statutes are read; we make no formal claim about our Z3 encoding.

The disagreement on top-level shape matters in practice. Catala's scope is a callable function: it takes inputs (facts about the case), returns outputs (the legal conclusion), and composes via scope-call sites. This is the right shape for tax code, where a tax-credit calculation is a function from taxpayer-facts to tax-owed. It is the wrong shape for a Penal Code section, where the section bundles definitions, elements, illustrations, and case law that are not function arguments. We discuss scope composition as a future-work layer (§10) that adds Catala-style scope-call semantics on top of Yuho's statute blocks rather than replacing them.

8.3 lam4

lam4 [29] originates from the SMU Centre for Computational Law and is, like Yuho, a Singapore-context legal DSL. It is the closest jurisdictional peer; the systems target different statute families (lam4: contracts and regulations; Yuho: penal code) but share an institutional context that makes comparison easy.

lam4's distinctive primitive is the named-norm reference and the `IS_INFRINGED` predicate: a contract clause can predicate on whether another named norm has been infringed, which composes cleanly into multi-clause reasoning. This composes directly into our planned G10 cross-section semantic resolver: a Yuho section like `statute 107 abetment` should be able to predicate on whether a different section's offence elements are satisfied, and the lam4 `IS_INFRINGED` shape is the right syntactic surface. We expect to

import the predicate name and semantics, with credit, when G10 lands.

8.4 LexScript

LexScript [6] is the closest tooling-shaped peer. It shares Yuho's compile-DSL-to-multiple-targets posture and structured top-level grammar (LexScript uses the `participle` parser generator in Go, where Yuho uses tree-sitter; the parser-generator shape is the same even though the host technologies differ). The substantive differences are in depth, not in approach: LexScript has a thinner element model (no burden qualifier, no causal-link edges, no Catala-style exception priority) and targets contracts rather than penal codes.

The most directly portable LexScript idea is the Tarjan-SCC analysis over the statute reference graph. We borrow it under G10: once the cross-section resolver lands, an SCC pass will catch unreachable exception branches and cyclic cross-references that the plain parser cannot detect. The classification of LexScript as a "thinner-grammar / broader-coverage" alternative to Yuho's "richer-grammar / single-family-coverage" is meant descriptively; the two systems make different bets on the right place to be on the expressivity-coverage tradeoff.

8.5 Akoma Ntoso and LegalRuleML

Akoma Ntoso [17] and LegalRuleML [18] are the OASIS-standardised XML schemas for, respectively, legislative structural markup and legal rule representation. They are not direct competitors to Yuho: their job is archival exchange and inter-system interoperability, not executable encoding.

The constructive constraint that these standards impose on Yuho is that we should be at least as faithful on the structural axis. Akoma Ntoso is the only competitor with first-class amendment-lineage support: its lifecycle model carries `<activeModifications>` and `<passiveModifications>` containers populated with `<textualMod>`, `<meaningMod>`, and `<scopeMod>` elements that capture every gazetted amendment with its own metadata. We treat that as the benchmark our `effective/repealed/subsumes/amends` clauses are obliged to match. Yuho's current model captures first-order temporal versioning (date-stamped clauses on the statute header) but not full textual-modification tracking in the `<textualMod>` sense; the `subsumes/amends` clauses approximate the latter at the section-level granularity that suffices for the Penal Code's amendment cadence. We expect to ship a Yuho-to-Akoma-Ntoso transpile target as future work; the converse direction is harder because Akoma Ntoso does not commit to a fixed shape for an offence definition, so an Akoma-Ntoso-to-Yuho lifting would have to reconstruct missing semantics from the markup.

LegalRuleML is more semantic than Akoma Ntoso: it has explicit deontic operators (`<lrml:Obligation>`, `<lrml:Prohibition>`, `<lrml:Permission>`), defeasibility markers (`<lrml:Override>`, `<lrml:Strength>`) that sit in the prioritised-default neighbourhood without being Catala-style prioritised rewriting, first-class burden-of-proof markup (`<lrml:hasReversedBurdenOfProof>` and related), partial temporal characteristics, and isomorphic mappings to several rule languages (Drools, Defeasible Logic via SPINdle in Governatori [12]). It does not have a notion of penalty algebra or illustration blocks. Where Yuho's deontic operators (`obligation` / `prohibition` / `permission` elements) overlap with LegalRuleML's, the syntactic mapping is straightforward; the underlying defeasibility machinery in Governatori et al. is richer than Yuho's static type-tag treatment, and a serious LegalRuleML-Yuho bridge would need to reconcile the two.

8.6 LDOC

LDOC [2] is a plain-text DSL for authoring professional documents that compiles to DOCX (TypeScript implementation; LSP server for editor integration). The source-of-truth posture matches Yuho's: a plain-text `.ldoc` file is canonical and the rendered DOCX is a downstream artefact. The systems differ on target audience and structural commitment — LDOC is a general- purpose document-authoring DSL where Yuho is statute-shaped — but the convergent decision to treat the plain-text source as canonical is direct evidence that the posture works in the legal-document neighbourhood.

The constructive borrowing from LDOC is the DOCX transpile target itself: shipping a clean DOCX rendering of a Yuho-encoded section (rather than treating Word as alien) makes the encoded artefact usable for the practitioner audience without changing the canonical source. The in-progress Microsoft Word add-in (§5.5) is meant to lift this further: a lawyer can browse Yuho-encoded sections from inside Word and insert formatted statute text via the add-in, while the `.yh` file in version control remains canonical.

8.7 Programming-languages context: language workbenches and the DSL economy

Yuho's architectural shape — a tree-sitter grammar feeding an immutable AST that several visitor-shaped consumers (transpilers, verifiers, linters, LSP, MCP) walk in parallel — is the shape of a *language workbench* in the sense of Erdweg et al. [8], who survey the state of the art across MPS, Spoofax, Rascal, Xtext, and others. The named-field grammar shape, the doc-comment-as-AST-attribute lift, and the multi-target transpilation pattern are all classic workbench moves. For a domain whose user is a lawyer, projectional editing (Völter [27]) is arguably a better surface than a text-based external DSL: a structured form on top of the same AST would give the verification benefits without the syntax-learning cost. We chose an external DSL because the encoder is the author and text-with-LSP suffices; an MPS-style projectional editor on the same AST is a credible alternative architecture and we do not claim ours is the only sensible one.

The decision to ship a one-way pipeline from `.yh` to DOCX/EN/JSON, rather than a bidirectional Word-to-yh editing loop, also has a literature: Foster et al.'s lens framework [10] formalises asymmetric vs. symmetric transformations and would let an authoring posture (round-trip Word↔`.yh`) coexist with Yuho's verification posture via a Word↔Yuho lens. We do not implement this; we note it as a credible alternative direction and a question we have not engaged.

The DSL-economy literature (Mernik, Heering & Sloane [16]) lays out the standard when-to-build-a-DSL criteria. Yuho clears the standard bars (productivity-per-section above the alternative, clear domain-specific surface, an audience that can read the syntax) for the encoder, but the audience claim has not been tested with

external users, and the DSL-economy literature would press on this point. We defer the validation to future work.

8.8 Wider context

Several lines of work outside the legal-DSL bracket inform Yuho's design without being direct comparators. Tree-sitter [5] provides the incremental parsing infrastructure that Yuho uses directly; the parser-generator-driven posture is also shared with LexScript (which picks `participle` in Go for the same reasons) and a growing number of other DSLs. The Language Server Protocol matured into the de-facto integration point between editors and language tooling, used by every modern legal DSL we surveyed. The Model Context Protocol [1], introduced in 2024, provides a parallel surface for AI clients; we are aware of no other legal DSL that ships an MCP server, though we expect several will in the next year.

The verification stack borrows from the broader formal-methods literature. Z3 [7] and Alloy [14] are well-established tools whose application to legal reasoning has been demonstrated since the 2000s; our use of them is engineering rather than research, and we make no claims of solver-side novelty.

9 Applicability beyond Singapore

The encoding presented in §7 is Singapore-only, and the paper is careful in §10 not to overstate portability without empirical evidence. This section makes the *positive* case: where the grammar plausibly transfers, what adaptation each candidate jurisdiction would require, and what empirical work would falsify the claim. We separate this from the limitations discussion deliberately. Disclaiming a thing and arguing for its plausibility are different rhetorical moves, and proof-of-concept readers reasonably want both.

9.1 The Anglo-Indian penal code family

The Singapore Penal Code 1871 is not a Singapore invention. It is the Straits Settlements adoption of the 1860 Indian Penal Code drafted by Macaulay's First Law Commission, lightly localised and subsequently amended. The same parent text underpins the criminal codes of India (IPC 1860, since 2023 reframed as the Bharatiya Nyaya Sanhita), Pakistan (PPC 1860), Bangladesh (PC 1860), Brunei (PC, Cap. 22), Malaysia (PC, Act 574), Sri Lanka (PC No. 2 of 1883), Myanmar (PC 1861), and historically several East African codes. The shared ancestry is not merely thematic. Section numbering, marginal-note conventions, illustration formatting, exception clauses, and the characteristic actus-reus / mens-rea / circumstance partitioning are recognisably the same drafting style across the family. A reader who has read Singapore's s415 will find the corresponding Indian and Malaysian s415 cosmetically familiar, often differing only in penalty quanta and amendment lineage.

This is what makes the family the natural test bed for grammar portability. Yuho's primitives were chosen to fit the drafting shape of penal statute — not the cultural specifics of one jurisdiction. The element partitioning is doctrinal, not local; the exception block is a Catala-style prioritised override pattern that maps directly onto the codified "provided that" clauses common to every member of the family; the penalty combinators capture the cumulative-or-alternative-or-conditional structure that the entire family inherits from Macaulay's draft. To the extent the grammar fits Singapore because the grammar fits Macaulay, the grammar should fit the others for the same reason.

9.2 What transfers, what adapts

We separate the grammar substrate (which we expect to transfer unchanged) from the per-jurisdiction configuration (which would need swapping).

Transfers as-is. The tree-sitter grammar, the AST, the five-block statute structure (`elements`, `penalty`, `exceptions`, `illustrations`, `caselaw`), the G10 reference resolver, the Z3 / Alloy verification scaffolding, the eight transpilers (including the OASIS Akoma Ntoso target and a Mermaid mindmap surface), and the entire editor surface (LSP, MCP, browser extension, explorer site, Word add-in) are jurisdiction-neutral. They operate on the AST, not on Singapore-specific tokens. Adopting a new jurisdiction would not require parser changes for these subsystems.

Adapts via configuration. A small number of surface features are configurable per jurisdiction:

- *Penalty vocabulary.* The set of admissible punishments varies: caning is in the Singapore, Malaysian, and Bruneian codes but not the Indian one; Pakistan retains *tazir* categories absent from Singapore; the Bangladeshi 2020 amendments enumerate specific aggravators absent elsewhere. Yuho's punishment kinds are a closed enum in the grammar; the per-jurisdiction adoption needs the enum extended (or restricted) and the controlled-English transpiler taught the new vocabulary.
- *Currency tokens.* Fines are encoded as a numeric quantum plus a currency tag (`SGD` for Singapore). Indian, Pakistani, and Bruneian codes use different currencies; the type system already admits the parameter, but the L2 fidelity linter expects the jurisdiction's reporting currency on initial scrape.
- *Statutory amendment lineage.* The `amends` clause points at an act number (e.g. "Act 32 of 2019"); each jurisdiction has its own amendment-act numbering scheme. The grammar accepts arbitrary strings here; only the L3 fidelity check that reconciles `amends` against the canonical scrape needs a jurisdiction-specific source.
- *Marginal-note style.* Older codes (1860–1900) place the marginal note above the section; newer ones (post-1990) inline it. The Phase A scraper's heuristic for extracting marginal notes is jurisdiction-specific; the AST and downstream tooling are not.

Requires substantive work. Two surfaces are not yet configuration-clean:

- *Mens-rea taxonomy.* Singapore distinguishes intention, knowledge, rashness, and negligence as distinct mental states; the 1860 IPC is identical, but the BNS 2023 (the recent Indian recodification) introduces collapsed categories that do not map one-to-one. A jurisdiction that uses a different taxonomy needs either a grammar extension or a vocabulary remapping pass.
- *The L3 reviewer's checklist.* The 11-point fidelity checklist (§5.4) bakes in conventions specific to the Singapore SSO

format (anchor IDs, marginal-note syntax, amendment notation). A jurisdiction with a different official publication format would need its own checklist; the parameterisation surface exists but has not been exercised.

9.3 Beyond the Penal Code family

The grammar is statute-shaped rather than penal-statute-shaped at the top level: a module contains statutes, a statute contains elements and penalties and exceptions, and the AST does not require the penalties block to be present. The question of whether Yuho fits non-penal statutes (tax, employment, intellectual property, competition law) reduces to whether those statutes share the *element + exception + illustration* shape. The Singapore Income Tax Act and the Employment Act both exhibit this shape on inspection of a sample of sections; the Companies Act does too, though with a heavier reliance on definitions blocks that Yuho already supports. A more rigorous claim awaits empirical encoding.

The shift to civil-law jurisdictions is more speculative. The French *Code pénal* and the German *Strafgesetzbuch* are similarly element-structured, but their definitional density and their reliance on cross-references to the civil-law general parts mean an encoding would lean heavily on Yuho's reference graph and less on its element partitioning. We do not claim the grammar fits civil-law codes; we claim the AST is the right level at which to test the question. Catala [15], focused on French tax law, is the closer analogue here, and a Yuho-Catala interop layer (scope composition over Yuho statute blocks) is the most natural extension if the civil-law direction is pursued.

9.4 An empirical path to validation

We commit to a concrete and falsifiable test of the portability claim. Encoding a 50-section sample of the Indian Penal Code is the minimum viable exercise: 50 sections is enough to surface grammar gaps that the Singapore exercise did not, and small enough to complete in a single encoder-quarter. The sampling strategy mirrors the Singapore encoding order: start with structurally simple property offences (theft, cheating, criminal misappropriation), move through bodily-harm hierarchies (hurt, grievous hurt, culpable homicide), and end with the procedural and abetment apparatus. Concretely, the test passes if (a) the grammar absorbs the 50-section sample at L1 with zero new gaps catalogued, (b) the L2 fidelity linters produce diagnostic densities within an order of magnitude of the Singapore baseline, and (c) the existing G1–G14 gap catalogue covers any drafting patterns the IPC sample exhibits but Singapore's PC does not. Failure on (a) implies a fifteenth gap and a grammar extension; failure on (b) implies the linters are over-fitted to Singapore conventions; failure on (c) implies the gap catalogue is not a stable artefact and would need a replication study before any portability claim is publishable.

The corpus directory layout is already jurisdiction-aware: the slim corpus, the canonical per-section JSON, and the reference graph are keyed on a `jurisdiction` field that defaults to Singapore but admits any string. Switching corpus base from Singapore to India requires no code change to the editor surfaces, only a parallel `library/penal_code_in/` directory and a per-jurisdiction `config.toml` for the per-jurisdiction parameters above. The build pipeline already supports running the corpus build over an arbitrary library root, so a multi-jurisdiction artefact is an engineering exercise on top of the empirical encoding work.

We frame this as future work in §12 rather than as a present-tense claim. What this section adds beyond §10 is a positive structural argument: the grammar substrate is jurisdiction-neutral, the configuration surface is small, and the empirical test is bounded. Portability has not been demonstrated, but the cost of demonstrating it is well-understood and the path to falsification is clear.

10 Limitations

This section makes the limitations explicit. We expect each to map onto a future-work line; some have concrete plans, others are deliberately out of scope. The threats to validity in §7.10 are methodological and concern the evaluation; the limitations here are structural and concern what Yuho can and cannot represent in principle.

Common-law doctrines are not modelled. Yuho captures the statute as drafted; it does not model judicial gloss. Doctrines like the *thin skull* rule, *novus actus interveniens*, or the mens-rea-inferred-from-act presumption live in case law and shape how the encoded elements apply in practice. Our `caselaw` block records the existence and citation of a relevant judicial interpretation, but does not lift the doctrine into the type system or the verification semantics. A holding that "mens rea may be inferred from the actus reus when the act is inherently dangerous" would, in a richer system, attach as a derived constraint that modifies the burden qualifier on the mens-rea element when the matching predicate fires. Yuho does not yet support this kind of case-law-to-type-system lifting; the closest current analogue is a documentation hint in the LSP hover output.

Single jurisdiction. The encoding is Singapore-only. The Anglo-Indian drafting heritage of the Singapore Penal Code (descended from Macaulay's 1860 Indian Penal Code draft via the 1872 SS Penal Code) means the Indian, Pakistani, Bangladeshi, Bruneian, and Malaysian codes share substantial structural overlap with Singapore's — but whether Yuho's grammar fits any of them is an empirical question we have not tested; we have no evidence of portability and should not be read as claiming any. The positive structural argument and the falsifiable empirical test are laid out separately in §9; what this paragraph claims is the absence of validation, not the absence of plausibility.

Shallow precedent integration. Case law is referenced by name, citation, and holding string. The holding is treated as opaque text, not parsed into structured elements. This is a deliberate choice for the proof-of-concept: lifting holdings into structured constraints would require either a separate case-law DSL or a substantial NLP pipeline, and either route risks fabricating structure that the holding text does not actually support. The cost of the choice is that queries like "which sections has a holding narrowed in the last decade?" cannot be answered from Yuho alone; the user must read the holding strings.

No live amendment ingestion. The canonical scrape lives in `library/penal_code/_raw/act.json` as a snapshot taken at scrape-time (the version embedded with the artefact reflects the

state on the dated commit in the project's git history). Amendments published subsequently by the Attorney-General's Chambers are not ingested automatically. A scheduled re-scrape with diff-against-prior detection is on the roadmap and is mostly an engineering exercise (the Phase A scraper already supports running over a fresh URL); we have not invested the time because the Penal Code's amendment cadence is annual at most and the cost of a manual re-scrape is small.

L3 reviewer is the encoder. The 11-point fidelity checklist the L3 reviewer applies (§5.4) is mechanical, but the same author wrote both the encodings and the dispatcher, and runs the checklist via agent sessions trained on encoding conventions the author defined. The L3 stamp should be read as "the encoder believes the encoding is faithful, and the mechanical checklist did not flag a fidelity issue", not as a legal opinion. External review by a Singapore-qualified lawyer is the natural next step for any encoding intended for use beyond proof of concept; we have not done it, and we acknowledge that a substantive sample of the L3-stamped encodings would benefit from independent inspection. The 524/524 L3 figure reported in §7.1 is therefore an *author-stamped* coverage statistic, not an externally-validated one: it reflects what the existing author-and-agent reviewer chain has stamped. Independent review by Singapore-qualified counsel is out of scope for this paper; that is a known limitation of the artefact, not a falsified claim.

SCC analysis is structural, not soundness. The reference-graph SCC pass (§7.3) flags non-trivial cycles in the implicit/subsumes cross-reference graph and surfaces them as `cross_section_cycle` warnings. A graph cycle is a finding, not a defect: two sections can mutually co-define each other in the statute as drafted, in which case the cycle is faithful and the warning is the diagnostic doing its job. Treating SCC findings as "unsound encodings" would be a category error; they are entry points for human review, the same posture as a fidelity diagnostic.

Verification scope. The Z3 and Alloy hookups (§5.3) target a deliberately narrow set of properties: exception-priority well-formedness, bounded element-combination enumeration, and pairwise inter-section conflict detection. Properties that require quantification over an unbounded number of historical events ("no defendant has ever been convicted under both s299 and s300 for the same act") are out of scope. Properties that require quantification over the universe of fact-patterns ("every act of deception that satisfies s415 also satisfies s420 if it induces property delivery") are reachable in principle but require a case-fact ontology that Yuho does not currently provide. We note this not as a flaw but as a calibration: the verification stack is for checking that the *encoding* is well-formed, not for adjudicating the *law*.

Soundness is fully mechanised at the conviction + cross-section + penalty layers. The mechanisation in `mechanisation/` (Lean 4, see §6.6) kernel-checks all five sub-lemmas of Theorem 6.1: 6.2 (element correspondence) and 6.4 (Catala-style exception precedence) in `Yuho/Soundness.lean` (v1); 6.3 (element-graph correspondence) in `Yuho/Graph.lean` (v2); 6.5 (cross-section composition) in `Yuho/Cross.lean` (v3); 6.6 (penalty correspondence) in `Yuho/Penalty.lean` (v3); plus the composed `full_conviction_correspondence` corollary in `Yuho/Graph.lean`.

The G8 / G14 unbounded-axis sentinels in `Penalty.lean` were closed in v4 (`fineUnlimited` / `caningUnspecified` constructors plus `Footprint` flag fields, kernel-checked); the cross-library case for `apply_scope` in `Cross.lean` was closed in v7 via the companion theorems `apply_scope_eval_none_of_extern` and `is_infringed_eval_none_of_extern`, which prove that extern references reduce to `none` on the operational side and therefore impose no SMT-side correspondence obligation. The case-law differential testing of §7.7 continues to use same-module fact patterns to keep the in-module correspondence non-trivial, but cross-library calls are no longer an unspoken caveat.

The v5 layer in `Yuho/Generator.lean` discharges the conviction-layer oracle assumption: the lemmas 6.2 / 6.3 / 6.4 now hold against a constructive `canonicalSMTModel` / `canonicalGraphModel` (built directly from the operational evaluator's outputs) rather than an axiomatic existential. The unconditional forms (`element_correspondence_unconditional`, etc.) are exported from `Yuho/Generator.lean`.

The v6 layer extends the discharge to the §6.5 penalty layer via `Generator.canonicalPenaltyModel` and to the single-statute case of the cross-section bundle via `Generator.canonicalCrossModel` + `canonical_cross_satisfies_singleton_singleton_exc`. The §6.5 strengthened bundle `PenaltySMTModel.satisfiesWF` requires the `Penalty.wellFormed` predicate (the linter's sentinel-propagation + non-empty-`orBoth` invariant, mechanised as a Bool-valued predicate in `Yuho/Penalty.lean`). The v6 cross-section refactor re-keys `excFires` on *qualified* atom names (`<sX>_exc_<label>_fires`) matching the Python generator's atom-naming convention, fixing the v3 raw-label collision when two statutes shared a label.

Residual oracle scope after v7: the claim that the Python `Z3Generator` implementation matches the Lean specification's emit shape is now discharged structurally on the full corpus. The generator-emit consolidation pass aligned the two emitters bicond-by-bicond on the conviction layer (Python now emits the leaf bicond family, the uniform `_fires` suffix on every exception, and the always-on conviction bicond), and `mechanisation/scripts/generate_fixtures.py` regenerates a Lean fixture file for every section in `library/penal_code/`.

The harness `make verify-structural-diff-full` runs `Generator.encodeStatute` and `Z3Generator._generate_statute_constraints` on parallel fixtures across all 524 sections and reports **524/524 matched, 0 divergent**, modulo two documented atom-naming canonicalisations (Lean's bare `<e.name>` leaf atom maps to Python's per-statute `<sX>_leaf_<e.name>`; the `_fires` bicond RHS uses an opaque placeholder on Lean and an inline priority-suppression expression on Python, both resolving to the same `Exception.firedSet` membership under the canonical model). `make verify-bulk-contrast` (§7.6) continues as a complementary behavioural cross-check against the operational evaluator on every encoded statute.

v7 also closed the multi-statute extension of the cross-section bundle: `canonical_cross_satisfies` kernel-checks for any

module under qualified-atom-name uniqueness + section-number uniqueness, both linter-enforced invariants. v8 mechanises the linter's cross-section-reference acyclicity invariant explicitly: `CrossRefGraph.acyclic` is a decidable Bool predicate over the module's surface-level `is_infringed` / `apply_scope` edges, and `acyclic_canonical_cross_satisfies` kernel-checks the discharge under that invariant. The discharge is vacuous at the current Lean abstraction level (the abstraction strips cross-section refs from `Statute`, so `Element.eval` never recurses through them); the hypothesis is preserved in the signature so a future v9 that lifts cross-section refs into `Element.eval` can rely on the invariant where it matters.

No formal grammar-completeness theorem. We claim that Yuho can express every Penal Code section at L1+L2 fidelity, backed by the 524/524 coverage figure (§7.1). This is empirical evidence rather than a proof: a future Penal Code amendment could in principle introduce a drafting pattern Yuho does not handle, in which case we would need to extend the grammar (the G1–G14 catalogue is what such extensions look like in practice). The notion of "grammar completeness for a statute family" is not something we know how to formalise in a way that survives drafting innovations; we report 524/524 as a coverage statistic rather than a soundness theorem.

11 Future work

Three concrete strands, plus a longer roadmap of explicitly deferred work.

11.1 Concrete next strands

The G10 cross-section resolver landed during the most recent iteration — the LexScript-borrowed Tarjan-SCC analysis (§7.3), the lam4-style `is_infringed` predicate, and the Catala-style `apply_scope` composition primitive are all shipped — so the remaining work in this strand is the elaboration of the evaluator-side semantics of `apply_scope` into a full element-graph executor that returns bindings, not just a structural "does this scope exist" predicate.

Cross-jurisdiction generalisation. 8 representative Indian Penal Code 1860 sections (s300 / s302 / s375 / s376 / s378 / s420 / s497 / s509) are encoded as a proof-of-concept demonstrating grammar fit, with a fully-scraped IPC corpus (493 sections) and a `yuho refs --compare-libraries` structural-overlap tool shipped alongside. Full IPC encoding (493 sections) and a richer SCC-overlap / amendment-divergence analysis are deferred to the agent-dispatch pipeline. The structural argument and the falsification criteria are laid out in §9; the Akoma Ntoso transpiler is the on-ramp for any cross-jurisdiction work that wants to interoperate with LegalDocML-consuming tooling.

Deeper precedent integration. Lifting case-law holding strings into structured constraints that modify element burden qualifiers when the matching predicate fires. Right now the L3 stamp records caselaw anchors as opaque strings; the next step is making them load-bearing.

Evaluator-side `apply_scope`. Elaborating the cross-section composition primitive into a full element-graph executor that returns bindings, not just a structural "does this scope exist" predicate.

11.2 Roadmap (Phase 2A / 2B / 2C)

The Yuho corpus and tooling extend along three orthogonal axes that we treat as deferred but explicit, not unscoped.

Phase 2A — diachronic depth. The Penal Code 1871 has 150+ years of amendments. Phase 2A extends every encoded section with every historical version since 1872; `yuho check --as-of 1995-06-15 my_facts.yh` becomes a first-class temporal query. Acceptance: 524 sections × at-least-3 historical versions encoded; per-section timeline rendering on `/s/<N>.html`; novel paper contributions on encoding lineage, an amendment-friction Levenshtein metric, and stable kernels classifying sections by drift since 1872.

Phase 2B — vision pivot. Yuho moves beyond the Penal Code into the wider Singapore criminal-justice corpus: Evidence Act 1893, Criminal Procedure Code, Constitution 1963, and Contract Law (common-law plus Misrepresentation Act and the Contracts (Rights of Third Parties) Act). Civil remedies and equitable doctrines need AST shapes Yuho does not yet have, so this is gated on grammar extension first.

Phase 2C — interactive learn-by-doing. The static explorer site grows into a guided learning surface for law students: per-section JS fact-builders, in-browser evaluators, worked-example walkthroughs from the canonical illustrations. Co-authoring with a law-school educator who can run a classroom trial unlocks measured learning-outcome evidence; the pedagogical claim is then "interactive structural pedagogy for common-law penal codes".

Phases 2A–2C are deliberately out of scope for the present artefact: 2A is a multi-month diachronic re-encode, 2B is a multi-statute scope expansion, and 2C is a co-authored pedagogical study. The current paper's empirical claims rest entirely on the static, current-version, single-Act corpus; the roadmap is provided for honest scoping rather than as load-bearing future work.

12 Conclusion

We presented Yuho, a statute-shaped domain-specific language, and used it to encode the entire Singapore Penal Code 1871: 524 sections, 524/524 passing parse-and-build at the L1+L2 coverage tiers, and 524 carrying a structured human-fidelity stamp at the strictest L3 tier (§7.1). The exercise surfaced a catalogue of fourteen grammar gaps (§3.9), distributed across ten genuine grammar limitations that we resolved in the parser and four diagnostic-layer issues that we resolved by adding fidelity linters. The result is a complete, queryable, machine-checkable encoding of a production penal code, plus the tooling — a tree-sitter parser, an immutable Python AST, eight transpilers (including Akoma Ntoso for cross-jurisdictional interop and a Mermaid mindmap surface), a reference-graph resolver with Tarjan-SCC analysis and the lam4-style `is_infringed` / Catala-style `apply_scope` cross-section predicates, a Z3-and-Alloy verification hookup, a Language Server, a Model Context Protocol server, and a VS Code extension — that turns the encoding into an executable artefact rather than a paper exercise.

The longer-term thesis is that *statute-as-source-of-truth* is a viable posture for legal infrastructure. This is the isomorphism [4] position with all the standard objections that come with it: clause-for-clause mirroring preserves traceability and makes amendments

propagate cleanly, but it forecloses some logic-friendly normalisations and inherits the open-textured-terms problem. We claim the isomorphism trade is the right one for penal code; we do not claim the standard objections have been dissolved. The encoded artefact is what the lawyer, the tooling, and the verification stack all read; downstream renderings (controlled English, LaTeX, Mermaid, DOCX) are generated, not authored. We do not claim that the 524 encoded sections constitute publication-ready legal advice; the limitations on common-law-doctrine modelling, precedent integration, and the author-administered L3 stamping (§10) are explicit about what the artefact is and is not. We do claim that the gap between a drafted penal section and an executable encoding is narrower than the existing literature suggests, and that an appropriately-shaped grammar dissolves a class of previously-reported encoding obstacles, leaving a residue that is tractable at full coverage for one statute family.

Artefact availability. The Yuho compiler, encoded library, Lean 4 mechanisation, and evaluation harness are available under an open-source licence at https://github.com/gongahkia/yuho. A versioned snapshot of this paper plus the v5.1.0 corpus is archived at Zenodo, DOI 10.5281/zenodo.19935537. The paper itself, including the build pipeline that injects coverage statistics into this very manuscript at compile time, is in the same repository under `paper/`.

References

[1] Anthropic. 2024. Model Context Protocol. https://modelcontextprotocol.io/.

[2] arismoko. 2026. LDOC: a plain-text DSL for authoring professional documents that compiles to DOCX. https://github.com/arismoko/LDOC. Project repository (TypeScript). Specification at `LDOC-V3-SPEC.md`; no peer-reviewed publication yet — the source tree is the canonical reference..

[3] Trevor Bench-Capon. 2003. Persuasion in practical argument using value-based argumentation frameworks. *Journal of Logic and Computation* 13, 3 (2003), 429–448.

[4] T. J. M. Bench-Capon and F. P. Coenen. 1992. Isomorphism and legal knowledge based systems. *Artificial Intelligence and Law* 1, 1 (1992), 65–86. doi:10.1007/BF00118480

[5] Max Brunsfeld. 2018. Tree-sitter: an incremental parsing system for programming tools. https://tree-sitter.github.io/tree-sitter/.

[6] codeNinja62. 2026. LexScript: a DSL compiler for structured contract logic. https://github.com/codeNinja62/LexScript. Project repository (MIT-licensed). v0.4 "Compiler Construction Project"; no peer-reviewed publication yet — the source tree is the canonical reference..

[7] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: an efficient SMT solver. In *TACAS*. 337–340.

[8] Sebastian Erdweg, Tijs van der Storm, Markus Völter, Laurence Tratt, Remi Bosman, William R. Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, Gabriël Konat, Pedro J. Molina, Martin Palatnik, Risto Pohjonen, Eugen Schindler, Klemens Schindler, Riccardo Solmi, Vlad Vergu, Eelco Visser, Kevin van der Vlist, Guido Wachsmuth, and Jimi van der Woning. 2015. Evaluating and comparing language workbenches: Existing results and benchmarks for the future. *Computer Languages, Systems & Structures* 44 (2015), 24–47. doi:10.1016/j.cl.2015.08.007

[9] Zhiwei Fei, Xiaoyu Shen, Dawei Zhu, Fengzhe Zhou, Zhuo Han, Songyang Zhang, Kai Chen, Zongwen Shen, and Jidong Ge. 2023. LawBench: Benchmarking legal knowledge of large language models. In *arXiv preprint arXiv:2309.16289*. Cited as positioning reference for LLM legal-reasoning benchmarks..

[10] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems* 29, 3 (2007), 17:1–17:65. doi:10.1145/1232420.1232424

[11] Dov Gabbay, John Horty, Xavier Parent, Ron van der Meyden, and Leendert van der Torre (Eds.). 2013. *Handbook of Deontic Logic and Normative Systems*. College Publications.

[12] Guido Governatori. 2005. Representing Business Contracts in RuleML. *International Journal of Cooperative Information Systems* 14, 2–3 (2005), 181–216. doi:10.1142/S0218843005001134

[13] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Chilton, Aditya Narayana, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel N. Rockmore, et al. 2023. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*. Cited as positioning reference for LLM legal-reasoning benchmarks..

[14] Daniel Jackson. 2002. Alloy: a lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology* 11, 2 (2002), 256–290. doi:10.1145/505145.505149

[15] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: a programming language for the law. In *Proceedings of the ACM on Programming Languages (ICFP)*, Vol. 5. doi:10.1145/3473582

[16] Marjan Mernik, Jan Heering, and Anthony M. Sloane. 2005. When and how to develop domain-specific languages. *Comput. Surveys* 37, 4 (2005), 316–344. doi:10.1145/1118890.1118892

[17] OASIS LegalDocML Technical Committee. 2018. Akoma Ntoso Version 1.0: OASIS standard for parliamentary, legislative, and judiciary documents. https://docs.oasis-open.org/legaldocml/akn-core/v1.0/akn-core-v1.0.html.

[18] OASIS LegalRuleML Technical Committee. 2021. LegalRuleML Core Specification Version 1.0: OASIS standard. https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/legalruleml-core-spec-v1.0.html.

[19] Object Management Group. 2022. Decision Model and Notation (DMN), Version 1.4. OMG Document Number: formal/2022-10-02, https://www.omg.org/spec/DMN/1.4/.

[20] Gabriel Zhe Mian Ong. 2026. *Yuho: A Domain-Specific Language for Encoding the Singapore Penal Code as Executable Statute*. doi:10.5281/zenodo.19935537 Open-source artefact (compiler, encoded library, Lean 4 mechanisation, evaluation harness). Versioned snapshot archived at Zenodo, https://doi.org/10.5281/zenodo.19935537..

[21] Henry Prakken and Giovanni Sartor. 2009. A logical analysis of burdens of proof. In *Legal Evidence and Proof: Statistics, Stories, Logic*, Hendrik Kaptein, Henry Prakken, and Bart Verheij (Eds.). Ashgate, 223–253.

[22] Marek Sergot. 1991. The Representation of Law in Computer Programs. In *Knowledge Based Systems and Legal Applications*, T. J. M. Bench-Capon (Ed.). Academic Press, 3–67.

[23] M. J. Sergot, F. Sadri, R. A. Kowalski, F. Kriwaczek, P. Hammond, and H. T. Cory. 1986. The British Nationality Act as a logic program. *Commun. ACM* 29, 5 (1986), 370–386. doi:10.1145/5689.5920

[24] Sepehr Sharifi, Alireza Parvizimosaed, Daniel Amyot, Luigi Logrippo, and John Mylopoulos. 2020. Symboleo: Towards a Specification Language for Legal Contracts. In *Proceedings of the IEEE 28th International Requirements Engineering Conference (RE)*. 364–369. doi:10.1109/RE48521.2020.00049

[25] Singapore Statutes Online, Attorney-General's Chambers. 1871, as amended. Penal Code 1871 (Singapore). https://sso.agc.gov.sg/Act/PC1871.

[26] L. Thomas van Binsbergen, Lu-Chi Liu, Robert van Doesburg, and Tom van Engers. 2020. eFLINT: a domain-specific language for executable norm specifications. In *Proceedings of the 19th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences (GPCE)*. 124–136. doi:10.1145/3425898.3426958

[27] Markus Völter. 2014. Towards user friendly transformation languages. In *Proceedings of the 3rd Workshop on the Globalization of Modeling Languages*. Background reference for projectional editing in MPS..

[28] Georg Henrik von Wright. 1951. Deontic Logic. *Mind* 60, 237 (1951), 1–15. doi:10.1093/mind/LX.237.1

[29] Seng Joe Watt, Oliver R. Goodenough, and Meng Weng Wong. 2023. Deontics and Time in Contracts: An Executable Semantics for the L4 DSL. In *Legal Knowledge and Information Systems — JURIX 2023: The Thirty-sixth Annual Conference, Maastricht, the Netherlands, 18–20 December 2023 (Frontiers in Artificial Intelligence and Applications, Vol. 379)*, Giovanni Sileno, Jerry Spanakis, and Gijs van Dijck (Eds.). IOS Press, 119–124. doi:10.3233/FAIA230954